Prastav: Low Level Design

Academic Project Workflow and Approval Management System

Document	Low Level Design for the whole system (M1 implemented, M2 planned)
Version	1.0, Milestone 1 submission
Date	19 October 2026
Repository	`prastav` (artifacts `prastav-backend`, `prastav-frontend`)
Base package	`com.prastav`
Stack	Java 17, Spring Boot 3.3, Spring Security (JWT), Spring Data JPA, Flyway, PostgreSQL 16; React 18 + TypeScript, Vite, TanStack Query, Axios

0. How to read this document

Every class, table, endpoint and diagram element carries one of two tags.

- **M1 Implemented**: exists in the submitted code with exactly the name used here.
- **M2 Planned**: designed now, built in Milestone 2. Nothing tagged M2 exists in the code. In diagrams, M2 elements are grey with a dashed border and the `<<M2>>` stereotype.

Names in this document, in `docs/diagrams.md` (Mermaid) and in `docs/uml/*.puml` (PlantUML) match the code exactly. The class catalogue in section 8 is generated from the Java sources, so field names, types and method signatures are copied rather than retyped.

1. Scope

1.1 Problem

Proposal submission, faculty review, revision and approval currently run over email, documents and spreadsheets. Nobody has a single view of a project's status, proposal versions get lost, feedback is not tied to the text it refers to, and there is no audit trail.

1.2 What Prastav does

One system for the full proposal lifecycle with an enforced workflow: only valid status transitions, performed only by the right role. Submitted proposal versions are immutable, every review points to the exact version it judged, and every important action is recorded in an append-only audit log.

1.3 Milestone split

Milestone	Scope	Status
M1, Core workflow (evaluation 19 to 23 Oct 2026)	FR-1 to FR-11: auth/JWT/roles, projects, immutable proposal versions, coordinator assignment, reviews, State-pattern workflow engine, Observer-based audit log, `allowedActions`, history endpoint, Flyway, seed data, Swagger, minimal React UI for three roles, unit and MockMvc tests	Implemented

Milestone	Scope	Status
M2, Completeness and quality (evaluation 16 to 20 Nov 2026)	FR-12 to FR-17: notifications (NotificationChannel strategy), version compare, reviewer reassignment, coordinator dashboard, audit-log viewer, filters/search/pagination; Docker images, GitHub Actions CI, Testcontainers integration tests, coverage	Designed here, not implemented

1.4 Non-goals (both milestones)

AI proposal analysis, ML risk prediction, GitHub integration, an external event bus, microservices, cloud deployment, real email delivery (the M2 email channel is a logging stub), and file uploads (proposals are structured text fields plus an optional link).

1.5 Roles

Role	Can do
STUDENT	Create a project, edit own draft, submit and resubmit, start a revision, view own projects, versions, feedback and history
FACULTY	View assigned projects, review the latest submitted version: approve, request revision or reject (comments mandatory for the last two)
COORDINATOR	View all projects, assign a faculty reviewer; M2: reassign, dashboard, audit viewer

2. Implementation status

2.1 Functional requirements

ID	Requirement	Milestone	Status	Where
FR-1	Register/login with JWT, role on account	M1	**M1 Implemented**	`AuthController`, `AuthService`, `JwtService`, `SecurityConfig`
FR-2	Student creates project with draft v1	M1	**M1 Implemented**	`ProjectService.createProject`, `ProposalVersionFactory.initialDraft`
FR-3	Draft editable only until submitted	M1	**M1 Implemented**	`ProposalService.updateDraft`, `ProposalVersion.updateContent`, DB trigger
FR-4	Submit makes the version immutable, project SUBMITTED	M1	**M1 Implemented**	`WorkflowService.submit`, `DraftState.submit`
FR-5	Coordinator assigns reviewer, project UNDER_REVIEW	M1	**M1 Implemented**	`WorkflowService.assignReviewer`, `SubmittedState.assignReviewer`
FR-6	Faculty approves / requests revision / rejects with comments	M1	**M1 Implemented**	`ReviewService.recordReview`, `ReviewDecisionHandler` x3, `UnderReviewState`
FR-7	Student sees feedback; revision creates vN+1 prefilled	M1	**M1 Implemented**	`ProposalService.createNextVersion`, `ProposalVersionFactory.nextDraftFrom`, UI detail page

ID	Requirement	Milestone	Status	Where
FR-8	Resubmission returns to UNDER_REVIEW with same reviewer	M1	**M1 Implemented**	`RevisionRequestedState.submit`
FR-9	Invalid transitions rejected with clear error	M1	**M1 Implemented**	`ProjectState` default methods, `IllegalTransitionException`, `GlobalExceptionHandler` (409)
FR-10	Audit log for every state change, assignment, submission, review	M1	**M1 Implemented**	`AuditEventListener`, `AuditLog`, append-only trigger
FR-11	Project timeline	M1 API / M2 UI	API **M1 Implemented**; UI tab **M2 Planned**	`ProjectHistoryService`, `GET /api/projects/{id}/history`
FR-12	Role-scoped lists with status filter, search, pagination	M2	Role scoping **M1 Implemented**; filter/search/page **M2 Planned**	`ProjectService.listProjects`; planned `ProjectSearchCriteria`, `ProjectSpecifications`
FR-13	In-app notifications	M2	**M2 Planned**	`NotificationListener`, `NotificationService`, `NotificationChannel`
FR-14	Compare two versions (field diff)	M2	**M2 Planned**	`VersionCompareService`, `VersionComparison`, `FieldDiff`
FR-15	Coordinator dashboard	M2	**M2 Planned**	`DashboardService`, `DashboardController`, `DashboardSummary`
FR-16	Audit-log viewer with filters	M2	**M2 Planned**	`AuditController`, `AuditLogFilter`, `AuditLogResponse`
FR-17	Reassign reviewer with audit entry	M2	**M2 Planned**	`WorkflowService.reassignReviewer`, `UnderReviewState.reassignReviewer`

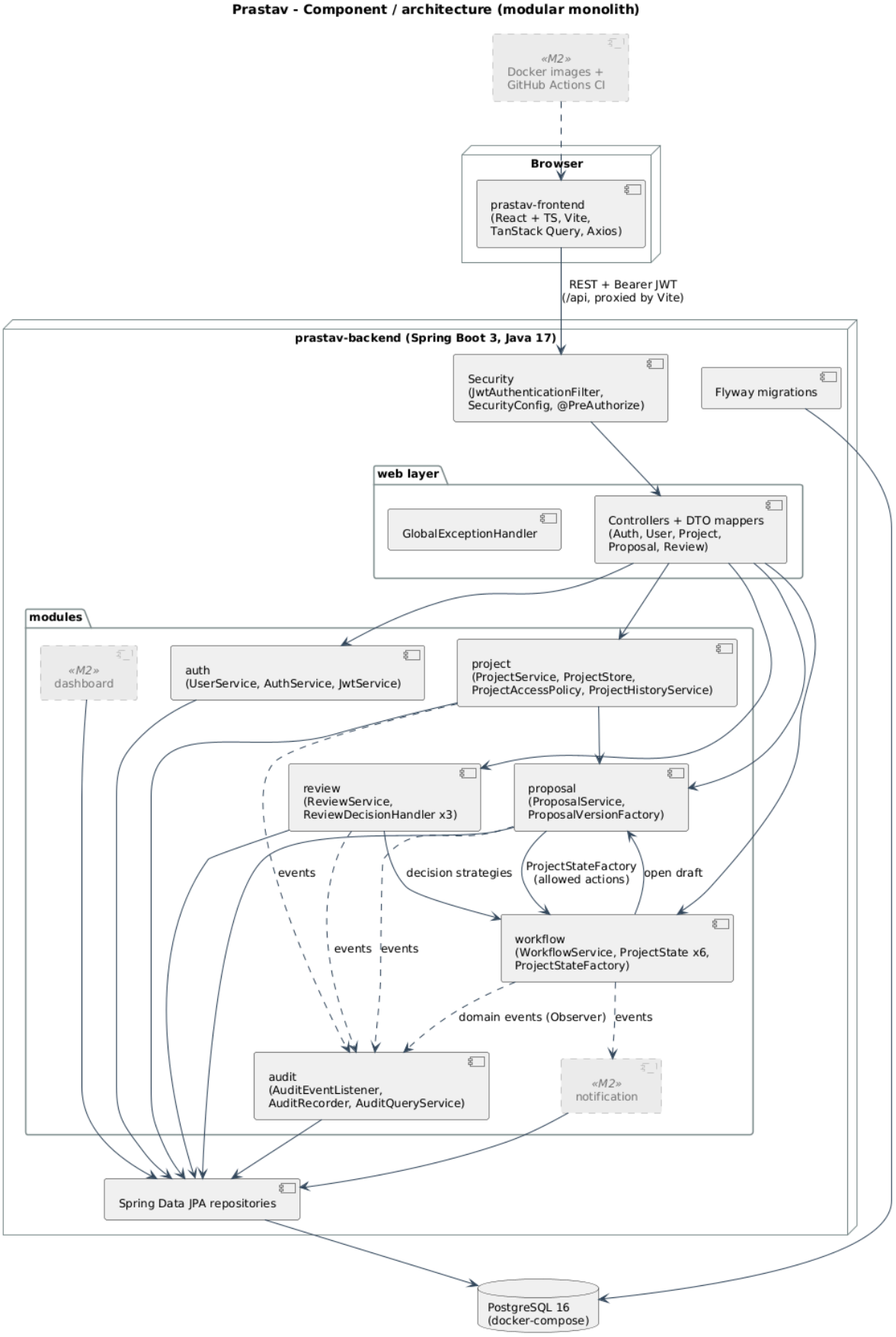
2.2 Engineering items

Item	Status	Evidence
Flyway schema with constraints and triggers	**M1 Implemented**	`backend/src/main/resources/db/migration/V1__init.sql`
Seed data (1 coordinator, 2 faculty, 3 students)	**M1 Implemented**	`DemoDataSeeder`
Optimistic locking on `Project`	**M1 Implemented**	`@Version Long version`, `CONCURRENT_UPDATE` mapping
Swagger UI with Bearer auth	**M1 Implemented**	`OpenApiConfig`, `/swagger-ui.html`
`allowedActions` in project detail	**M1 Implemented**	`AllowedActionsResolver`, `ProjectDetailResponse.allowedActions`
Unit tests: every state, WorkflowService, ReviewService, versioning	**M1 Implemented**	60 tests in `backend/src/test/java` (see section 16)

Item	Status	Evidence
MockMvc tests (401 / 403 / 409 mapping)	M1 Implemented	`ProjectControllerWebTest` (3 tests)
React UI for three roles	M1 Implemented	`frontend/src`
PostgreSQL via docker-compose	M1 Implemented	`docker-compose.yml` (database only)
Backend/frontend Dockerfiles, compose for whole stack	M2 Planned	section 15.3
GitHub Actions CI (build, test, lint), coverage report	M2 Planned	section 15.3
Testcontainers integration tests incl. concurrent submit	M2 Planned	section 16.2

3. Architecture

Prastav is a **modular monolith**: one Spring Boot deployable and one PostgreSQL database. Each module owns its entities and repository; other modules reach it only through its public service classes (`UserService`, `ProjectStore`, `ProposalService`, `ReviewService`, `AuditQueryService`, `WorkflowService`) or by listening to its domain events.


Prastav - Component / architecture (modular monolith)
«M2»
Docker images +
GitHub Actions CI
Browser
prastav-frontend
(React + TS, Vite,
TanStack Query, Axios)
REST + Bearer JWT
(/api, proxied by Vite)
prastav-backend (Spring Boot 3, Java 17)
Security
(JwtAuthenticationFilter,
SecurityConfig, @PreAuthorize)
Flyway migrations
web layer
GlobalExceptionHandler
Controllers + DTO mappers
(Auth, User, Project,
Proposal, Review)
modules
«M2»
dashboard
auth
(UserService, AuthService, JwtService)
project
(ProjectService, ProjectStore,
ProjectAccessPolicy, ProjectHistoryService)
review
(ReviewService,
ReviewDecisionHandler x3)
proposal
(ProposalService,
ProposalVersionFactory)
events
decision strategies
ProjectStateFactory
(allowed actions)
open draft
events
events
workflow
(WorkflowService, ProjectState x6,
ProjectStateFactory)
domain events (Observer)
events
audit
(AuditEventListener,
AuditRecorder, AuditQueryService)
«M2»
notification
Spring Data JPA repositories
PostgreSQL 16
(docker-compose)

3.1 Layers inside a module

Layer	Contains	Rule
web	`*Controller`, request/response records, `*Mapper`	HTTP only: parse, validate (`@Valid`), call one service, map to DTO. No business rules.
service	`*Service`, `ProjectStore`, `ProjectAccessPolicy`, `AllowedActionsResolver`, decision handlers	Use cases and transactions (`@Transactional`). Authorization layer 2.
domain	entities, enums, value records, state classes, `Guards`, events	Business rules. State classes and entities are plain Java and tested without Spring.
repository	`*Repository extends JpaRepository`	Persistence only; used inside its own module.

3.2 Module dependencies (no cycles between beans)

```
auth      <- (everyone needs Actor/User)        common <- everyone
workflow  -> project (ProjectStore), proposal (ProposalService), auth (UserService)
proposal  -> project (ProjectStore, ProjectAccessPolicy), workflow (ProjectStateFactory)
review    -> workflow (WorkflowService via handlers), proposal, project, auth
project   -> proposal (ProposalService), workflow (AllowedActionsResolver), auth, audit (history read)
audit     -> listens to events from project, proposal, workflow, review
```

`ProjectStore` exists so that `ProposalService` and `WorkflowService` can load and save projects without depending on `ProjectService`, which itself depends on `ProposalService`; this keeps constructor injection free of cycles.

3.3 Request lifecycle

1. `JwtAuthenticationFilter` verifies the Bearer token and puts an `Actor(userId, role)` in the security context.
2. `@PreAuthorize` on the controller method checks the role (authorization layer 1).
3. The controller calls a service with the `Actor` as an explicit parameter.
4. The service loads data, checks ownership/assignment via `ProjectAccessPolicy` or the state guards (layer 2), changes state, saves, publishes events.
5. Event listeners run synchronously in the same transaction (audit rows commit or roll back with the change).
6. The controller maps the result to a response record. Any exception becomes an `ApiError` in `GlobalExceptionHandler` or `RestAuthErrorHandler`.

4. Package structure

```
com.prastav
├─ PrastavApplication                                        [M1]
├─ common      ApiError, FieldErrorDetail, ErrorCode, PrastavException, NotFoundException,
│              ForbiddenException, ValidationException, ConflictException,
│              UnauthenticatedException, BaseEntity, GlobalExceptionHandler, OpenApiConfig   [M1]
├─ auth        Role, User, Actor, AuthResult, UserRepository, UserService, AuthService,
│              JwtService, JwtAuthenticationFilter, RestAuthErrorHandler, SecurityConfig,
│              DemoDataSeeder                                                                [M1]
│  └─ web      AuthController, UserController, RegisterRequest, LoginRequest, AuthResponse,
│              UserResponse, UserSummary, UserMapper                                         [M1]
├─ workflow    ProjectStatus, Action, ProjectState, DraftState, SubmittedState,
│              UnderReviewState, RevisionRequestedState, ApprovedState, RejectedState,
│              ProjectStateFactory, Guards, AllowedActionsResolver, IllegalTransitionException,
│              ProjectStatusChangedEvent, WorkflowService                                    [M1]
├─ project     Project, ProjectRepository, ProjectStore, ProjectAccessPolicy, ProjectService,
│              NewProject, ProjectView, ProjectCreatedEvent, ProjectHistoryService,
│              TimelineEntry, TimelineEntryType                                              [M1]
│              ProjectSearchCriteria, ProjectSpecifications                                  [M2]
│  └─ web      ProjectController, CreateProjectRequest, AssignReviewerRequest,
│              ProjectSummaryResponse, ProjectDetailResponse, ProjectMapper                  [M1]
│              ReassignReviewerRequest                                                       [M2]
├─ proposal    ProposalVersion, ProposalVersionState, ProposalContent, ProposalVersionFactory,
│              ProposalVersionRepository, ProposalService, ProposalVersionCreatedEvent,
│              ProposalDraftUpdatedEvent                                                     [M1]
│              VersionCompareService, VersionComparison, FieldDiff                           [M2]
│  └─ web      ProposalController, ProposalVersionResponse, ProposalMapper                  [M1]
├─ review      Review, ReviewDecision, ReviewRepository, ReviewService, ReviewDecisionHandler,
│              ApproveDecisionHandler, RequestRevisionDecisionHandler, RejectDecisionHandler,
│              ReviewRecordedEvent                                                           [M1]
│  └─ web      ReviewController, ReviewRequest, ReviewResponse, ReviewMapper                [M1]
├─ audit       AuditLog, AuditAction, AuditLogRepository, AuditRecorder, JpaAuditRecorder,
│              AuditEventListener, AuditQueryService                                         [M1]
│              AuditController, AuditLogFilter, AuditLogResponse                            [M2]
├─ notification Notification, NotificationType, NotificationRepository, NotificationChannel,
│              InAppNotificationChannel, EmailNotificationChannel, NotificationService,
│              NotificationListener, NotificationController, NotificationResponse           [M2]
└─ dashboard   DashboardService, DashboardController, DashboardSummary, ReviewerWorkload     [M2]
```

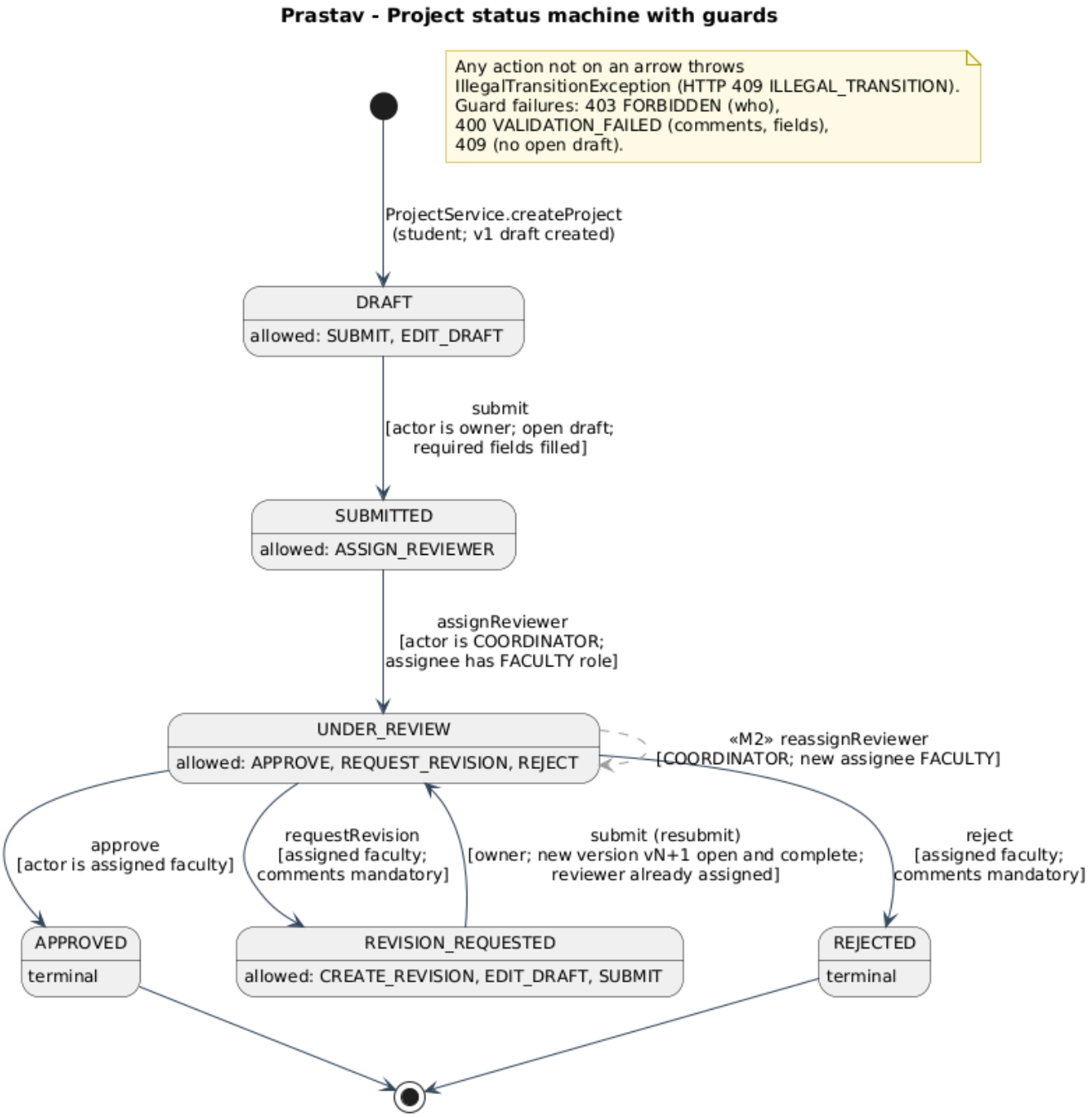
5. Workflow engine

5.1 Status table (the only legal transitions)

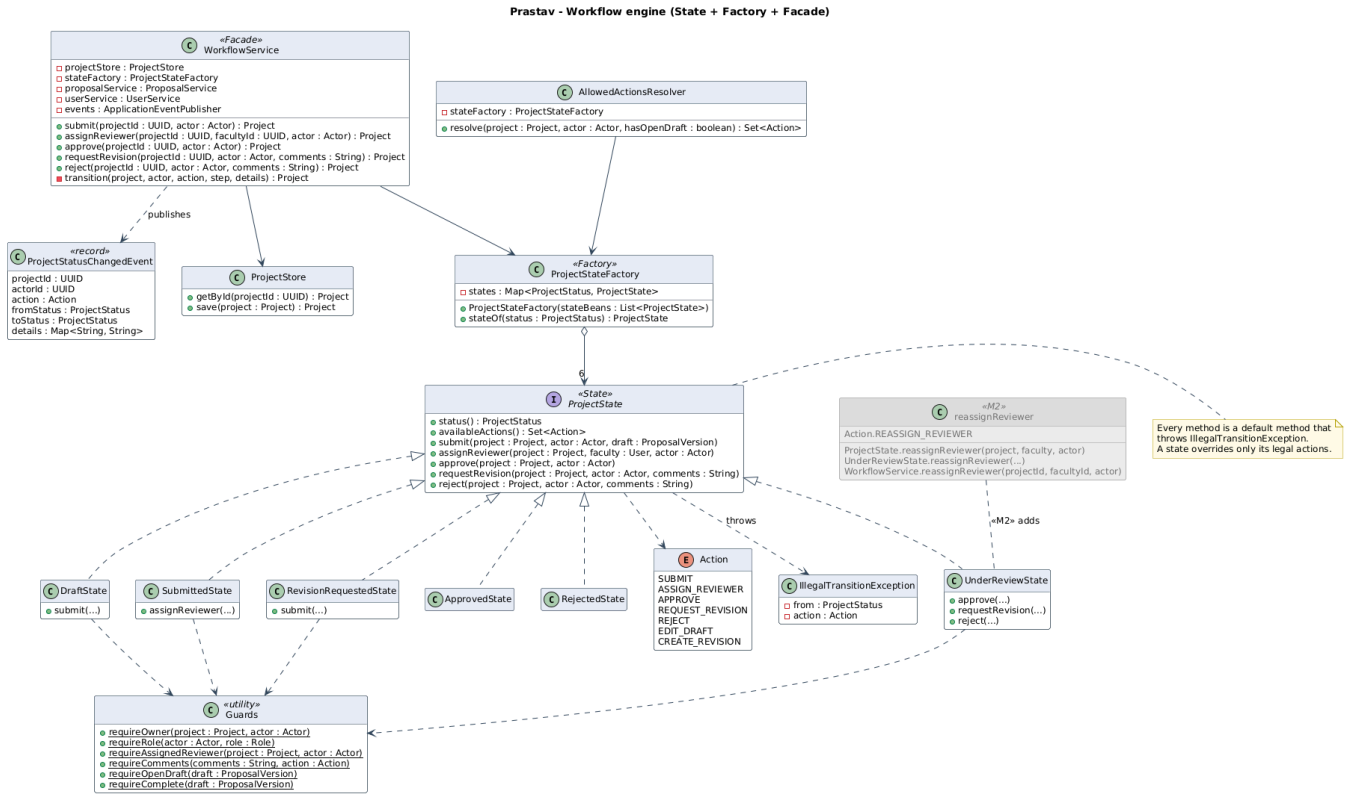
From	Action	Actor	To	Guard (implemented in)
DRAFT	`submit`	Student owner	SUBMITTED	owner; open draft exists; required fields filled (`DraftState.submit` via `Guards`)
SUBMITTED	`assignReviewer`	Coordinator	UNDER_REVIEW	actor is COORDINATOR; assignee has FACULTY role (`SubmittedState.assignReviewer`)
UNDER_REVIEW	`approve`	Assigned faculty	APPROVED	actor is the assigned reviewer (`UnderReviewState.approve`); review is stored against the latest submitted version (`ReviewService`)
UNDER_REVIEW	`requestRevision`	Assigned faculty	REVISION_REQUESTED	assigned reviewer; comments mandatory
UNDER_REVIEW	`reject`	Assigned faculty	REJECTED	assigned reviewer; comments mandatory

From	Action	Actor	To	Guard (implemented in)
REVISION_REQUESTED	`submit` (resubmit)	Student owner	UNDER_REVIEW	owner; new version is open and complete; reviewer already assigned (`RevisionRequestedState.submit`)
UNDER_REVIEW	`reassignReviewer`	Coordinator	UNDER_REVIEW	**M2 Planned** COORDINATOR; new assignee FACULTY and different from current

APPROVED and REJECTED are terminal. Any pair not in this table throws `IllegalTransitionException` and returns HTTP 409 `ILLEGAL_TRANSITION`.



5.2 State pattern classes



`ProjectState` **M1 Implemented** declares five transition methods as **default methods that throw** `IllegalTransitionException(status(), action)`. Each concrete state overrides only what is legal from it:

State class	Overrides	`availableActions()`
`DraftState`	`submit`	SUBMIT, EDIT_DRAFT
`SubmittedState`	`assignReviewer`	ASSIGN_REVIEWER
`UnderReviewState`	`approve`, `requestRevision`, `reject`	APPROVE, REQUEST_REVISION, REJECT
`RevisionRequestedState`	`submit` (resubmit)	CREATE_REVISION, EDIT_DRAFT, SUBMIT
`ApprovedState`	nothing (terminal)	none
`RejectedState`	nothing (terminal)	none

`DraftState.submit` and `RevisionRequestedState.submit` share a name but behave differently (SUBMITTED versus UNDER_REVIEW). That polymorphism is what replaces `if (status == ...)` chains.

States are stateless Spring `@Component`s. `ProjectStateFactory` **M1 Implemented** receives `List<ProjectState>` by constructor injection, builds an `EnumMap<ProjectStatus, ProjectState>`, and fails at startup if a status has no state or two states claim one status. Adding a status therefore means one enum value plus one class.

`Guards` **M1 Implemented** holds the static precondition checks used by the states (`requireOwner`, `requireRole`, `requireAssignedReviewer`, `requireComments`, `requireOpenDraft`, `requireComplete`). Keeping them static and dependency-free lets tests construct states with `new DraftState()`.

5.3 `WorkflowService` algorithm (Facade)

Every public method (`submit`, `assignReviewer`, `approve`, `requestRevision`, `reject`) is `@Transactional` and follows the same private template `transition(project, actor, action, step, details)`:

1. Load the project through `ProjectStore.getById` (404 if missing). For `submit`, also load the open draft through `ProposalService.findOpenDraft`; for `assignReviewer`, load the faculty user through `UserService.getById`.
2. Remember `from = project.getStatus()`.
3. `stateFactory.stateOf(from)` and call the action on it. The state runs its guards, then mutates the project (`moveTo`, `assignReviewer`) and, for submit, freezes the draft (`ProposalVersion.submit()`).
4. `projectStore.save(project)`. The `@Version` column makes a concurrent second transition fail with `OptimisticLockingFailureException`, mapped to 409 `CONCURRENT_UPDATE`.
5. Publish `ProjectStatusChangedEvent(projectId, actorId, action, from, to, details)`.

If step 3 throws, steps 4 and 5 never run and the transaction rolls back, so an illegal action leaves no trace except the error response.

5.4 `allowedActions`

`AllowedActionsResolver.resolve(project, actor, hasOpenDraft)` **M1 Implemented** starts from `stateOf(status).availableActions()` and keeps an action only if this actor may perform it:

Action	Kept when
SUBMIT, EDIT_DRAFT	actor owns the project and an open draft exists
CREATE_REVISION	actor owns the project and no open draft exists
ASSIGN_REVIEWER	actor is COORDINATOR
APPROVE, REQUEST_REVISION, REJECT	actor is FACULTY and the assigned reviewer

The UI renders buttons only from this list. The backend still re-checks every action, so a hand-crafted request gets the same 403/409 as before.

`ProposalService` uses the same `availableActions()` to decide whether EDIT_DRAFT or CREATE_REVISION is legal in the current status, so proposal rules and workflow rules cannot drift apart.

6. Events and listeners (Observer)

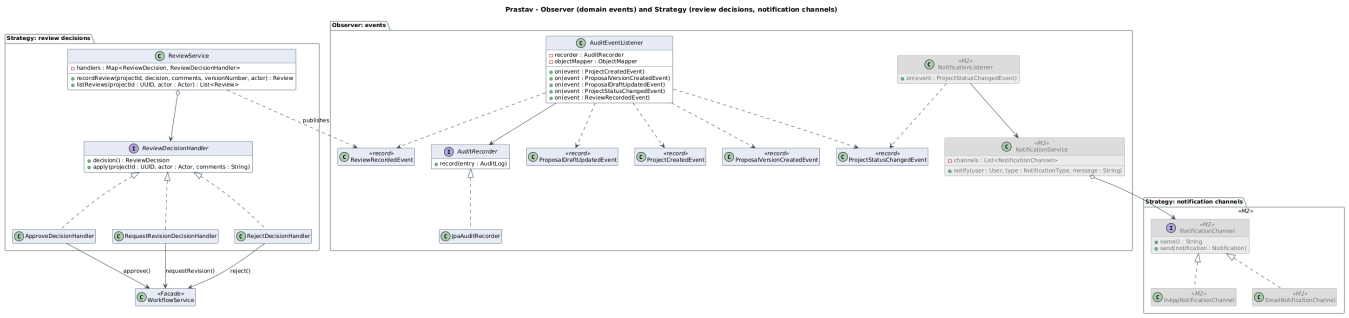
Publishers use Spring's `ApplicationEventPublisher`; listeners use `@EventListener`. Listeners run **synchronously in the publisher's transaction**: an audit row is committed if and only if the change it describes is committed.

Event (record) **M1 Implemented**	Published by	Payload	`AuditEventListener` writes
`ProjectCreatedEvent`	`ProjectService.createProject`	projectId, actorId, title	`PROJECT_CREATED`, to_status DRAFT
`ProposalVersionCreatedEvent`	`ProposalService.createInitialDraft`, `createNextVersion`	projectId, versionNumber, actorId	`VERSION_CREATED`
`ProposalDraftUpdatedEvent`	`ProposalService.updateDraft`	projectId, versionNumber, actorId	`DRAFT_UPDATED`
`ProjectStatusChangedEvent`	`WorkflowService` (every transition)	projectId, actorId, action, fromStatus, toStatus, details	`PROPOSAL_SUBMITTED` (SUBMIT), `REVIEWER_ASSIGNED` (ASSIGN_REVIEWER), otherwise `STATUS_CHANGED`; from/to filled

Event (record) M1 Implemented	Published by	Payload	`AuditEventListener` writes
`ReviewRecordedEvent`	`ReviewService.recordReview`	projectId, reviewId, versionNumber, reviewerId, decision	`REVIEW_RECORDED`

`AuditEventListener` depends on the one-method `AuditRecorder` interface (implemented by `JpaAuditRecorder`), not on a repository. `details` is a small JSON object built with Jackson, e.g. `{"versionNumber":"2","action":"SUBMIT"}`.

M2 Planned **`NotificationListener`** subscribes to `ProjectStatusChangedEvent` and calls `NotificationService.notify(user, type, message)`. Rules: REVIEWER_ASSIGNED notifies the assigned faculty; APPROVE/REQUEST_REVISION/REJECT notify the student owner (type REVIEW_DECISION); a resubmission (REVISION_REQUESTED to UNDER_REVIEW) notifies the reviewer (type RESUBMITTED). It will be annotated `@TransactionalEventListener(phase = AFTER_COMMIT)` because, unlike audit, a notification must not be sent for a change that rolls back. No workflow class changes.



7. Review decision strategies (Strategy)

`ReviewDecisionHandler` M1 Implemented: `ReviewDecision decision()` and `void apply(UUID projectId, Actor actor, String comments)`.

Implementation	Decision	Calls
`ApproveDecisionHandler`	APPROVE	`WorkflowService.approve(projectId, actor)`
`RequestRevisionDecisionHandler`	REQUEST_REVISION	`WorkflowService.requestRevision(projectId, actor, comments)`
`RejectDecisionHandler`	REJECT	`WorkflowService.reject(projectId, actor, comments)`

`ReviewService` receives `List<ReviewDecisionHandler>` and indexes it into an `EnumMap`. `recordReview` runs in this order:

1. Pick the handler for the decision.
2. Normalise comments (trim; blank becomes null).
3. Load the latest **submitted** version. If the request carries `versionNumber` and it is not the latest, return 409 (stale-page protection, business rule 4).
4. `handler.apply(...)`: the workflow transition with all guards (assigned reviewer, status, mandatory comments).
5. Save `Review(project, latestVersion, reviewer, decision, comments)` and publish `ReviewRecordedEvent`.

Because step 4 runs before step 5 inside one transaction, a failed transition never leaves a review row behind.

M2 Planned **`NotificationChannel`** strategy: `String name()` and `void send(Notification notification)`. `InAppNotificationChannel` saves a `notifications` row; `EmailNotificationChannel` is a stub that only logs (real email is a non-goal). `NotificationService` receives `List<NotificationChannel>` and sends through each, so a new channel is a new `@Component` class.

8. Class catalogue

Generated from `backend/src/main/java` (96 classes). Private helper methods are omitted; private fields are listed. Every class in 8.1 to 8.12 is M1 Implemented. Section 8.13 lists M2 Planned classes.

8.1 Package `com.prastav`

`PrastavApplication` M1 Implemented

class · `src/main/java/com/prastav/PrastavApplication.java`

Responsibility. Entry point of the Prastav modular monolith.

Methods:

- `public static void main(String[] args)`

8.2 Package `com.prastav.common`

`ApiError` M1 Implemented

record · `src/main/java/com/prastav/common/ApiError.java`

Responsibility. The single error body used by every endpoint: { timestamp, status, code, message, path, fieldErrors[] }.

Components: `Instant timestamp, int status, ErrorCode code, String message, String path, List<FieldErrorDetail> fieldErrors`

Methods:

- `public static ApiError of(ErrorCode code, String message, String path, List<FieldErrorDetail> fieldErrors)`

`BaseEntity` M1 Implemented

abstract class `implements Persistable<UUID>` · @MappedSuperclass · `src/main/java/com/prastav/common/BaseEntity.java`

Responsibility. Common identity for all entities: an application-assigned UUID and a creation timestamp. Ids are generated in the constructor, so domain objects have an identity before they are saved (which keeps unit tests free of a database). Implementing Persistable tells Spring Data that a freshly constructed entity is new, so save() performs an INSERT instead of a merge.

Field	Type	Visibility
`id`	`UUID`	private

Field	Type	Visibility
`createdAt`	`Instant`	private
`newEntity`	`boolean`	private

Methods:

- `protected BaseEntity()`
- `public UUID getId()`
- `public Instant getCreatedAt()`
- `public boolean isNew()`
- `protected void markNotNew()`
- `public boolean equals(Object other)`
- `public int hashCode()`

`ConflictException` M1 Implemented

class `extends PrastavException` · `src/main/java/com/prastav/common/ConflictException.java`

Responsibility. The operation conflicts with the current state of the data, outside the status table itself, e.g. editing a submitted proposal version or opening a second draft (409 ILLEGAL_TRANSITION).

Methods:

- `public ConflictException(String message)`

`ErrorCode` M1 Implemented

enum · `src/main/java/com/prastav/common/ErrorCode.java`

Responsibility. Stable, machine-readable error codes returned in ApiError#code(). Each code carries the HTTP status it maps to, so the mapping lives in one place.

Constants: `VALIDATION_FAILED`, `UNAUTHENTICATED`, `FORBIDDEN`, `NOT_FOUND`, `ILLEGAL_TRANSITION`, `CONCURRENT_UPDATE`, `INTERNAL_ERROR`

Field	Type	Visibility
`httpStatus`	`int`	private

Methods:

- `public int httpStatus()`

`FieldErrorDetail` M1 Implemented

record · `src/main/java/com/prastav/common/FieldErrorDetail.java`

Responsibility. One invalid request field, reported inside ApiError#fieldErrors().

Components: `String field, String message`

`ForbiddenException` M1 Implemented

class `extends PrastavException` · `src/main/java/com/prastav/common/ForbiddenException.java`

Responsibility. The caller is authenticated but not allowed to act on this resource (403 FORBIDDEN).

Methods:

- `public ForbiddenException(String message)`

`GlobalExceptionHandler` M1 Implemented

class · @RestControllerAdvice · `src/main/java/com/prastav/common/GlobalExceptionHandler.java`

Responsibility. Translates exceptions into the uniform ApiError body. Controllers contain no try/catch; every failure path ends here, which keeps the error contract identical across endpoints. Stack traces are logged, never returned.

Field	Type	Visibility
`log`	`static Logger`	private

Methods:

- `public ResponseEntity<ApiError> handleBusiness(PrastavException ex, HttpServletRequest request)`
- `public ResponseEntity<ApiError> handleInvalidBody(MethodArgumentNotValidException ex, HttpServletRequest request)`
- `public ResponseEntity<ApiError> handleMalformed(Exception ex, HttpServletRequest request)`
- `public ResponseEntity<ApiError> handleAccessDenied(AccessDeniedException ex, HttpServletRequest request)`
- `public ResponseEntity<ApiError> handleConcurrentUpdate(OptimisticLockingFailureException ex, HttpServletRequest request)`
- `public ResponseEntity<ApiError> handleNoResource(NoResourceFoundException ex, HttpServletRequest request)`
- `public ResponseEntity<ApiError> handleUnexpected(Exception ex, HttpServletRequest request)`

`NotFoundException` M1 Implemented

class `extends PrastavException` · `src/main/java/com/prastav/common/NotFoundException.java`

Responsibility. A requested entity does not exist (404 NOT_FOUND).

Methods:

- `public NotFoundException(String entityName, Object id)`

`OpenApiConfig` M1 Implemented

class · @Configuration · `src/main/java/com/prastav/common/OpenApiConfig.java`

Responsibility. Swagger UI metadata and the "Authorize" button for Bearer JWTs.

`PrastavException` M1 Implemented

abstract class `extends RuntimeException` · `src/main/java/com/prastav/common/PrastavException.java`

Responsibility. Base class for all expected business errors. The ErrorCode decides the HTTP status, so GlobalExceptionHandler needs only one handler for the whole hierarchy.

Field	Type	Visibility
`code`	`ErrorCode`	private

Methods:

- `protected PrastavException(ErrorCode code, String message)`
- `public ErrorCode getCode()`

`UnauthenticatedException` M1 Implemented

class `extends PrastavException` · `src/main/java/com/prastav/common/UnauthenticatedException.java`

Responsibility. Login failed or credentials are missing (401 UNAUTHENTICATED).

Methods:

- `public UnauthenticatedException(String message)`

`ValidationException` M1 Implemented

class `extends PrastavException` · `src/main/java/com/prastav/common/ValidationException.java`

Responsibility. A business validation rule failed, e.g. mandatory review comments missing (400 VALIDATION_FAILED).

Methods:

- `public ValidationException(String message)`

8.3 Package `com.prastav.auth`

`Actor` M1 Implemented

record · `src/main/java/com/prastav/auth/Actor.java`

Responsibility. The authenticated caller, rebuilt from the JWT on every request and passed explicitly into services. Passing the actor as a parameter (instead of reading a static security context deep inside the domain) keeps services and state classes testable without Spring Security.

Components: `UUID userId, Role role`

Methods:

- `public boolean hasRole(Role expected)`

`AuthResult` M1 Implemented

record · `src/main/java/com/prastav/auth/AuthResult.java`

Responsibility. Result of a successful register or login: the signed JWT and the user it belongs to.

Components: `String token, User user`

`AuthService` M1 Implemented

class · @Service · `src/main/java/com/prastav/auth/AuthService.java`

Responsibility. Registration and login. Passwords are hashed with BCrypt; a successful call returns a JWT.

Field	Type	Visibility
`userRepository`	`UserRepository`	private
`passwordEncoder`	`PasswordEncoder`	private
`jwtService`	`JwtService`	private

Methods:

- `public AuthService(UserRepository userRepository, PasswordEncoder passwordEncoder, JwtService jwtService)`
- `public AuthResult register(String name, String email, String password, Role role)`
- `public AuthResult login(String email, String password)`

`DemoDataSeeder` M1 Implemented

class `implements ApplicationRunner` · @Component · `src/main/java/com/prastav/auth/DemoDataSeeder.java`

Responsibility. Creates the demo accounts (1 coordinator, 2 faculty, 3 students) on an empty database. Done in Java rather than SQL so that passwords are hashed with the same BCrypt encoder the app uses.

Field	Type	Visibility
`log`	`static Logger`	private
`userRepository`	`UserRepository`	private
`passwordEncoder`	`PasswordEncoder`	private

Methods:

- `public DemoDataSeeder(UserRepository userRepository, PasswordEncoder passwordEncoder)`
- `public void run(ApplicationArguments args)`

`JwtAuthenticationFilter` M1 Implemented

class `extends OncePerRequestFilter` · `src/main/java/com/prastav/auth/JwtAuthenticationFilter.java`

Responsibility. Reads Authorization: Bearer , verifies it and stores the Actor as the principal with authority ROLE_. An invalid token simply leaves the request unauthenticated; the entry point then answers 401. Created inside SecurityConfig, not as a standalone bean.

Field	Type	Visibility
`BEARER_PREFIX`	`static String`	private
`jwtService`	`JwtService`	private

Methods:

- `public JwtAuthenticationFilter(JwtService jwtService)`
- `protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain chain)`

`JwtService` M1 Implemented

class · @Service · `src/main/java/com/prastav/auth/JwtService.java`

Responsibility. Issues and verifies HS256 JWTs carrying two claims: userId and role. Tokens expire after prastav.jwt.expiration-minutes (60 by default).

Field	Type	Visibility
`CLAIM_USER_ID`	`static String`	private
`CLAIM_ROLE`	`static String`	private
`key`	`SecretKey`	private
`expiry`	`Duration`	private

Methods:

- `public JwtService(@Value("$`
- `public String generateToken(User user)`
- `public Optional<Actor> parse(String token)`

`RestAuthErrorHandler` M1 Implemented

class `implements AuthenticationEntryPoint, AccessDeniedHandler` · @Component · `src/main/java/com/prastav/auth/RestAuthErrorHandler.java`

Responsibility. Writes 401/403 responses raised inside the security filter chain in the standard ApiError format.

Field	Type	Visibility
`objectMapper`	`ObjectMapper`	private

Methods:

- `public RestAuthErrorHandler(ObjectMapper objectMapper)`
- `public void commence(HttpServletRequest request, HttpServletResponse response, AuthenticationException authException) throws IOException`
- `public void handle(HttpServletRequest request, HttpServletResponse response, AccessDeniedException accessDeniedException) throws IOException`

`Role` M1 Implemented

enum · `src/main/java/com/prastav/auth/Role.java`

Responsibility. The three roles of Prastav. Stored as a string on the user row and as a JWT claim.

Constants: `STUDENT`, `FACULTY`, `COORDINATOR`

`SecurityConfig` M1 Implemented

class · @Configuration · `src/main/java/com/prastav/auth/SecurityConfig.java`

Responsibility. Stateless JWT security. Layer 1 of authorization: every request except login/register/Swagger must be authenticated, and @PreAuthorize on controllers gates by role. Layer 2 (ownership and assignment) lives in the services and state classes.

Field	Type	Visibility
`jwtService`	`JwtService`	private
`errorHandler`	`RestAuthErrorHandler`	private

Field	Type	Visibility
`allowedOrigin`	`String`	private

Methods:

- `public SecurityConfig(JwtService jwtService, RestAuthErrorHandler errorHandler, @Value("$`
- `public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception`
- `public CorsConfigurationSource corsConfigurationSource()`
- `public PasswordEncoder passwordEncoder()`

`User` M1 Implemented

class `extends BaseEntity` · @Entity · `src/main/java/com/prastav/auth/User.java`

Responsibility. A person who can log in. The role is fixed on the account and copied into the JWT.

Field	Type	Visibility
`name`	`String`	private
`email`	`String`	private
`passwordHash`	`String`	private
`role`	`Role`	private

Methods:

- `protected User()`
- `public User(String name, String email, String passwordHash, Role role)`
- `public Actor toActor()`
- `public String getName()`
- `public String getEmail()`
- `public String getPasswordHash()`
- `public Role getRole()`

`UserRepository` M1 Implemented

interface `extends JpaRepository<User, UUID>` · `src/main/java/com/prastav/auth/UserRepository.java`

Responsibility. Repository pattern: persistence for User, used only inside the auth module.

Methods:

- `Optional<User> findByEmail(String email)`
- `boolean existsByEmail(String email)`
- `List<User> findByRoleOrderByNameAsc(Role role)`
- `List<User> findAllByOrderByNameAsc()`

`UserService` M1 Implemented

class · @Service · `src/main/java/com/prastav/auth/UserService.java`

Responsibility. The auth module's public service for other modules. Other modules never touch UserRepository directly; they ask this service.

Field	Type	Visibility
`userRepository`	`UserRepository`	private

Methods:

- `public UserService(UserRepository userRepository)`
- `public User getById(UUID id)`
- `public List<User> listByRole(Role role)`
- `public Map<UUID, String> namesOf(Collection<UUID> ids)`

8.4 Package `com.prastav.auth.web`

`AuthController` M1 Implemented

class · @RestController · `src/main/java/com/prastav/auth/web/AuthController.java`

Responsibility. Register, login and "who am I". HTTP only; all rules are in AuthService.

Field	Type	Visibility
`authService`	`AuthService`	private
`userService`	`UserService`	private

Methods:

- `public AuthController(AuthService authService, UserService userService)`
- `public AuthResponse register(RegisterRequest request)`
- `public AuthResponse login(LoginRequest request)`
- `public UserResponse me(Actor actor)`

`AuthResponse` M1 Implemented

record · `src/main/java/com/prastav/auth/web/AuthResponse.java`

Responsibility. Returned by register and login: the JWT plus the user it was issued for.

Components: `String token, UserResponse user`

`LoginRequest` M1 Implemented

record · `src/main/java/com/prastav/auth/web/LoginRequest.java`

Responsibility. Body of POST /api/auth/login.

Components: `String email, String password`

`RegisterRequest` M1 Implemented

record · `src/main/java/com/prastav/auth/web/RegisterRequest.java`

Responsibility. Body of POST /api/auth/register. COORDINATOR is rejected by the service.

Components: `String name, String email, String password, Role role`

`UserController` M1 Implemented

class · @RestController · `src/main/java/com/prastav/auth/web/UserController.java`

Responsibility. Lets the coordinator list users, e.g. ?role=FACULTY to pick a reviewer.

Field	Type	Visibility
`userService`	`UserService`	private

Methods:

- `public UserController(UserService userService)`
- `public List<UserResponse> list(Role role)`

`UserMapper` M1 Implemented

final class · `src/main/java/com/prastav/auth/web/UserMapper.java`

Responsibility. DTO mapper: entities never leave the service layer as-is.

Methods:

- `public static UserResponse toResponse(User user)`
- `public static UserSummary toSummary(User user)`
- `public static AuthResponse toAuthResponse(AuthResult result)`

`UserResponse` M1 Implemented

record · `src/main/java/com/prastav/auth/web/UserResponse.java`

Responsibility. Public view of a user; never contains the password hash.

Components: `UUID id, String name, String email, Role role`

`UserSummary` M1 Implemented

record · `src/main/java/com/prastav/auth/web/UserSummary.java`

Responsibility. Compact user reference embedded in other responses (owner, reviewer).

Components: `UUID id, String name`

8.5 Package `com.prastav.workflow`

`Action` M1 Implemented

enum · `src/main/java/com/prastav/workflow/Action.java`

Responsibility. Everything a user can do to a project. The first five are status transitions handled by ProjectState; EDIT_DRAFT and CREATE_REVISION are proposal operations whose availability still depends on the status, so each state lists them too. The UI shows buttons from this list.

Constants: `SUBMIT`, `ASSIGN_REVIEWER`, `APPROVE`, `REQUEST_REVISION`, `REJECT`, `EDIT_DRAFT`, `CREATE_REVISION`

`AllowedActionsResolver` M1 Implemented

class · @Component · `src/main/java/com/prastav/workflow/AllowedActionsResolver.java`

Responsibility. Computes allowedActions for the project detail response: the actions legal in the current state, filtered to the ones this actor may perform. The UI only shows buttons from this set; the backend still re-checks every action when it is called.

Field	Type	Visibility
`stateFactory`	`ProjectStateFactory`	private

Methods:

- `public AllowedActionsResolver(ProjectStateFactory stateFactory)`
- `public Set<Action> resolve(Project project, Actor actor, boolean hasOpenDraft)`

`ApprovedState` M1 Implemented

class `implements ProjectState` · @Component · `src/main/java/com/prastav/workflow/ApprovedState.java`

Responsibility. State pattern: APPROVED. Terminal; inherits the "illegal" default for every action.

Methods:

- `public ProjectStatus status()`
- `public Set<Action> availableActions()`

`DraftState` M1 Implemented

class `implements ProjectState` · @Component · `src/main/java/com/prastav/workflow/DraftState.java`

Responsibility. State pattern: DRAFT. The owner may edit the draft and submit it once all required fields are filled. Submitting freezes the version (it becomes immutable) and moves the project to SUBMITTED.

Methods:

- `public ProjectStatus status()`
- `public Set<Action> availableActions()`
- `public void submit(Project project, Actor actor, ProposalVersion draft)`

`Guards` M1 Implemented

final class · `src/main/java/com/prastav/workflow/Guards.java`

Responsibility. Transition preconditions shared by the state classes. Static and dependency-free so that state objects stay plain Java and can be tested with new DraftState().

Methods:

- `public static void requireOwner(Project project, Actor actor)`
- `public static void requireRole(Actor actor, Role role)`
- `public static void requireAssignedReviewer(Project project, Actor actor)`
- `public static void requireComments(String comments, Action action)`
- `public static void requireOpenDraft(ProposalVersion draft)`
- `public static void requireComplete(ProposalVersion draft)`

`IllegalTransitionException` M1 Implemented

class `extends PrastavException` · `src/main/java/com/prastav/workflow/IllegalTransitionException.java`

Responsibility. Thrown when an action is not legal in the project's current status (409 ILLEGAL_TRANSITION).

Field	Type	Visibility
`from`	`ProjectStatus`	private
`action`	`Action`	private

Methods:

- `public IllegalTransitionException(ProjectStatus from, Action action)`
- `public ProjectStatus getFrom()`
- `public Action getAction()`

`ProjectState` M1 Implemented

interface · `src/main/java/com/prastav/workflow/ProjectState.java`

Responsibility. State pattern. One implementation per ProjectStatus. Every action defaults to "illegal" and a state overrides only the actions it allows, so the transition table lives in six small classes instead of a growing switch statement. Why: adding a status means adding one class (Open/Closed), each state's rules can be unit-tested in isolation, and the same call (e.g. submit) behaves differently per state through polymorphism rather than if/else on the status.

Methods:

- `ProjectStatus status()`
- `Set<Action> availableActions()`
- `default void submit(Project project, Actor actor, ProposalVersion draft)`
- `default void assignReviewer(Project project, User faculty, Actor actor)`
- `default void approve(Project project, Actor actor)`
- `default void requestRevision(Project project, Actor actor, String comments)`
- `default void reject(Project project, Actor actor, String comments)`

`ProjectStateFactory` M1 Implemented

class · @Component · `src/main/java/com/prastav/workflow/ProjectStateFactory.java`

Responsibility. Factory pattern. Central place that turns a ProjectStatus into its ProjectState object. Why: callers never instantiate states or switch on the status. Spring injects every ProjectState bean, so a new status is registered by adding its class; the constructor fails fast at startup if any status has no state or two states claim the same status.

Field	Type	Visibility
`states`	`Map<ProjectStatus, ProjectState>`	private

Methods:

- `public ProjectStateFactory(List<ProjectState> stateBeans)`
- `public ProjectState stateOf(ProjectStatus status)`

`ProjectStatus` M1 Implemented

enum · `src/main/java/com/prastav/workflow/ProjectStatus.java`

Responsibility. The six lifecycle statuses of a project. APPROVED and REJECTED are terminal.

Constants: `DRAFT`, `SUBMITTED`, `UNDER_REVIEW`, `REVISION_REQUESTED`, `APPROVED`, `REJECTED`

Methods:

- `public boolean isTerminal()`

`ProjectStatusChangedEvent` M1 Implemented

record · `src/main/java/com/prastav/workflow/ProjectStatusChangedEvent.java`

Responsibility. Observer pattern event, published by WorkflowService after every successful transition. The workflow does not know who listens (today: the audit listener).

Components: `UUID projectId, UUID actorId, Action action, ProjectStatus fromStatus, ProjectStatus toStatus, Map<String, String> details`

`RejectedState` M1 Implemented

class `implements ProjectState` · @Component · `src/main/java/com/prastav/workflow/RejectedState.java`

Responsibility. State pattern: REJECTED. Terminal; inherits the "illegal" default for every action.

Methods:

- `public ProjectStatus status()`
- `public Set<Action> availableActions()`

`RevisionRequestedState` M1 Implemented

class `implements ProjectState` · @Component · `src/main/java/com/prastav/workflow/RevisionRequestedState.java`

Responsibility. State pattern: REVISION_REQUESTED. The owner creates version N+1, edits it and resubmits. submit here goes straight back to UNDER_REVIEW with the same reviewer: the same method name as in DraftState but different behaviour, chosen by polymorphism rather than by an if on the status.

Methods:

- `public ProjectStatus status()`
- `public Set<Action> availableActions()`
- `public void submit(Project project, Actor actor, ProposalVersion draft)`

`SubmittedState` M1 Implemented

class `implements ProjectState` · @Component · `src/main/java/com/prastav/workflow/SubmittedState.java`

Responsibility. State pattern: SUBMITTED. Waiting for the coordinator to assign a FACULTY reviewer.

Methods:

- `public ProjectStatus status()`
- `public Set<Action> availableActions()`
- `public void assignReviewer(Project project, User faculty, Actor actor)`

`UnderReviewState` **M1 Implemented**

class `implements ProjectState` · @Component · `src/main/java/com/prastav/workflow/UnderReviewState.java`

Responsibility. State pattern: UNDER_REVIEW. Only the assigned faculty member may decide. Requesting a revision or rejecting requires comments; approving does not.

Methods:

- `public ProjectStatus status()`
- `public Set<Action> availableActions()`
- `public void approve(Project project, Actor actor)`
- `public void requestRevision(Project project, Actor actor, String comments)`
- `public void reject(Project project, Actor actor, String comments)`

`WorkflowService` **M1 Implemented**

class · @Service · `src/main/java/com/prastav/workflow/WorkflowService.java`

Responsibility. Facade pattern. The single entry point for every status change: load project, get its state from ProjectStateFactory, run the action, save, publish ProjectStatusChangedEvent. Controllers and other services never set a status themselves. Why: callers see five simple methods instead of coordinating states, repositories and events, and every transition goes through the same audited path in one transaction.

Field	Type	Visibility
`projectStore`	`ProjectStore`	private
`stateFactory`	`ProjectStateFactory`	private
`proposalService`	`ProposalService`	private
`userService`	`UserService`	private
`events`	`ApplicationEventPublisher`	private

Methods:

- `public WorkflowService(ProjectStore projectStore, ProjectStateFactory stateFactory, ProposalService proposalService, UserService userService, ApplicationEventPublisher events)`
- `public Project submit(UUID projectId, Actor actor)`
- `public Project assignReviewer(UUID projectId, UUID facultyId, Actor actor)`
- `public Project approve(UUID projectId, Actor actor)`
- `public Project requestRevision(UUID projectId, Actor actor, String comments)`
- `public Project reject(UUID projectId, Actor actor, String comments)`

8.6 Package `com.prastav.project`

`NewProject` **M1 Implemented**

record · `src/main/java/com/prastav/project/NewProject.java`

Responsibility. Metadata of a project being created (the proposal fields travel separately as ProposalContent).

Components: `String title, String description, String techStack, String teamMembers`

`Project` M1 Implemented

class `extends BaseEntity` · @Entity · `src/main/java/com/prastav/project/Project.java`

Responsibility. Aggregate root of the workflow: project metadata, owner, assigned reviewer and current status. #moveTo and #assignReviewer are called only by the workflow state classes; @Version gives optimistic locking so two simultaneous transitions cannot both succeed.

Field	Type	Visibility
`title`	`String`	private
`description`	`String`	private
`techStack`	`String`	private
`teamMembers`	`String`	private
`owner`	`User`	private
`reviewer`	`User`	private
`status`	`ProjectStatus`	private
`version`	`Long`	private
`updatedAt`	`Instant`	private

Methods:

- `protected Project()`
- `public Project(String title, String description, String techStack, String teamMembers, User owner)`
- `public void moveTo(ProjectStatus next)`
- `public void assignReviewer(User faculty)`
- `public boolean isOwnedBy(UUID userId)`
- `public boolean isReviewedBy(UUID userId)`
- `public String getTitle()`
- `public String getDescription()`
- `public String getTechStack()`
- `public String getTeamMembers()`
- `public User getOwner()`
- `public User getReviewer()`
- `public ProjectStatus getStatus()`
- `public Long getVersion()`
- `public Instant getUpdatedAt()`

`ProjectAccessPolicy` M1 Implemented

class · @Component · `src/main/java/com/prastav/project/ProjectAccessPolicy.java`

Responsibility. Layer 2 of authorization for reads and proposal edits: the coordinator sees everything, a student sees own projects, faculty see projects assigned to them. (Transition rules are checked by the workflow state classes.)

Methods:

- `public boolean canView(Project project, Actor actor)`
- `public void requireView(Project project, Actor actor)`
- `public void requireOwner(Project project, Actor actor)`
- `public void requireRole(Actor actor, Role role)`

`ProjectCreatedEvent` M1 Implemented

record · `src/main/java/com/prastav/project/ProjectCreatedEvent.java`

Responsibility. Observer pattern event: a student created a project (recorded as PROJECT_CREATED in the audit log).

Components: `UUID projectId, UUID actorId, String title`

`ProjectHistoryService` M1 Implemented

class · @Service · `src/main/java/com/prastav/project/ProjectHistoryService.java`

Responsibility. Builds the merged timeline (FR-11): version creations, review decisions and status changes, sorted by time. Status changes come from the audit log, so the timeline and the audit trail agree.

Field	Type	Visibility
`projectStore`	`ProjectStore`	private
`accessPolicy`	`ProjectAccessPolicy`	private
`proposalService`	`ProposalService`	private
`reviewService`	`ReviewService`	private
`auditQueryService`	`AuditQueryService`	private
`userService`	`UserService`	private

Methods:

- `public ProjectHistoryService(ProjectStore projectStore, ProjectAccessPolicy accessPolicy, ProposalService proposalService, ReviewService reviewService, AuditQueryService auditQueryService, UserService userService)`
- `public List<TimelineEntry> history(UUID projectId, Actor actor)`

`ProjectRepository` M1 Implemented

interface `extends JpaRepository<Project, UUID>` · `src/main/java/com/prastav/project/ProjectRepository.java`

Responsibility. Repository pattern: persistence for Project. Role-scoped list queries live here.

Methods:

- `List<Project> findByOwner_IdOrderByUpdatedAtDesc(UUID ownerId)`
- `List<Project> findByReviewer_IdOrderByUpdatedAtDesc(UUID reviewerId)`
- `List<Project> findAllByOrderByUpdatedAtDesc()`

`ProjectService` M1 Implemented

class · @Service · `src/main/java/com/prastav/project/ProjectService.java`

Responsibility. Use cases for creating, listing and viewing projects. Status changes go through WorkflowService.

Field	Type	Visibility
`projectRepository`	`ProjectRepository`	private
`projectStore`	`ProjectStore`	private
`accessPolicy`	`ProjectAccessPolicy`	private
`proposalService`	`ProposalService`	private
`allowedActionsResolver`	`AllowedActionsResolver`	private
`userService`	`UserService`	private
`events`	`ApplicationEventPublisher`	private

Methods:

- `public ProjectService(ProjectRepository projectRepository, ProjectStore projectStore, ProjectAccessPolicy accessPolicy, ProposalService proposalService, AllowedActionsResolver allowedActionsResolver, UserService userService, ApplicationEventPublisher events)`
- `public ProjectView createProject(NewProject newProject, ProposalContent proposal, Actor actor)`
- `public List<Project> listProjects(Actor actor)`
- `public ProjectView getProject(UUID projectId, Actor actor)`

`ProjectStore` M1 Implemented

class · @Component · `src/main/java/com/prastav/project/ProjectStore.java`

Responsibility. The project module's load/save port for other modules (workflow, proposal, review). They depend on this class instead of on ProjectRepository, which keeps the module boundary and avoids a constructor cycle with ProjectService.

Field	Type	Visibility
`projectRepository`	`ProjectRepository`	private

Methods:

- `public ProjectStore(ProjectRepository projectRepository)`
- `public Project getById(UUID projectId)`
- `public Project save(Project project)`

`ProjectView` M1 Implemented

record · `src/main/java/com/prastav/project/ProjectView.java`

Responsibility. Read model for the project detail screen: the project, its newest proposal version, the number of the latest submitted version and the actions this actor may take now.

Components: `Project project, ProposalVersion currentProposal, Integer latestSubmittedVersion, Set<Action> allowedActions`

`TimelineEntry` M1 Implemented

record · `src/main/java/com/prastav/project/TimelineEntry.java`

Responsibility. One line of the project history (FR-11). A read model, not an entity.

Components: `Instant at, TimelineEntryType type, UUID actorId, String actorName, String summary, Integer versionNumber`

`TimelineEntryType` M1 Implemented

enum · `src/main/java/com/prastav/project/TimelineEntryType.java`

Responsibility. Kinds of entries in the merged project history.

Constants: `VERSION_CREATED`, `STATUS_CHANGED`, `REVIEW`

8.7 Package `com.prastav.project.web`

`AssignReviewerRequest` M1 Implemented

record · `src/main/java/com/prastav/project/web/AssignReviewerRequest.java`

Responsibility. Body of POST /api/projects/{id}/assign.

Components: `UUID facultyId`

`CreateProjectRequest` M1 Implemented

record · `src/main/java/com/prastav/project/web/CreateProjectRequest.java`

Responsibility. Body of POST /api/projects: project metadata plus the (possibly partial) first draft.

Components: `String title, String description, String techStack, String teamMembers, ProposalContent proposal`

Methods:

- `public NewProject toNewProject()`
- `public ProposalContent proposalOrEmpty()`

`ProjectController` M1 Implemented

class · @RestController · `src/main/java/com/prastav/project/web/ProjectController.java`

Responsibility. Project endpoints, including the two workflow actions triggered from the project page. HTTP only.

Field	Type	Visibility
`projectService`	`ProjectService`	private
`workflowService`	`WorkflowService`	private
`projectHistoryService`	`ProjectHistoryService`	private

Methods:

- `public ProjectController(ProjectService projectService, WorkflowService workflowService, ProjectHistoryService projectHistoryService)`
- `public ProjectDetailResponse create(CreateProjectRequest request, Actor actor)`
- `public List<ProjectSummaryResponse> list(Actor actor)`

- `public ProjectDetailResponse get(UUID id, Actor actor)`
- `public ProjectDetailResponse submit(UUID id, Actor actor)`
- `public ProjectDetailResponse assign(UUID id, AssignReviewerRequest request, Actor actor)`
- `public List<TimelineEntry> history(UUID id, Actor actor)`

`ProjectDetailResponse` M1 Implemented

record · `src/main/java/com/prastav/project/web/ProjectDetailResponse.java`

Responsibility. Project detail including the current proposal and allowedActions for the caller.

Components: `UUID id, String title, String description, String techStack, String teamMembers, ProjectStatus status, UserSummary owner, UserSummary reviewer, ProposalVersionResponse currentProposal, Integer latestSubmittedVersion, List<Action> allowedActions, Long version, Instant createdAt, Instant updatedAt`

`ProjectMapper` M1 Implemented

final class · `src/main/java/com/prastav/project/web/ProjectMapper.java`

Responsibility. DTO mapper for projects.

Methods:

- `public static ProjectSummaryResponse toSummary(Project project)`
- `public static ProjectDetailResponse toDetail(ProjectView view)`

`ProjectSummaryResponse` M1 Implemented

record · `src/main/java/com/prastav/project/web/ProjectSummaryResponse.java`

Responsibility. One row of a project list.

Components: `UUID id, String title, ProjectStatus status, UserSummary owner, UserSummary reviewer, Instant updatedAt`

8.8 Package `com.prastav.proposal`

`ProposalContent` M1 Implemented

record · `src/main/java/com/prastav/proposal/ProposalContent.java`

Responsibility. The editable proposal fields as one value. Used as the request body for draft edits and copied into a new version when a revision starts. Fields may be blank while drafting; completeness is checked on submit.

Components: `String problemStatement, String objectives, String methodology, String expectedOutcome, String timeline, String referenceLink`

Methods:

- `public static ProposalContent empty()`

`ProposalDraftUpdatedEvent` M1 Implemented

record · `src/main/java/com/prastav/proposal/ProposalDraftUpdatedEvent.java`

Responsibility. Observer pattern event: the owner saved changes to the open draft.

Components: `UUID projectId, int versionNumber, UUID actorId`

`ProposalService` M1 Implemented

class · @Service · `src/main/java/com/prastav/proposal/ProposalService.java`

Responsibility. Proposal versioning use cases: edit the open draft, start a revision, list versions. Whether editing or revising is allowed in the current status is asked of the project's ProjectState, so these rules stay consistent with the workflow table.

Field	Type	Visibility
`versionRepository`	`ProposalVersionRepository`	private
`versionFactory`	`ProposalVersionFactory`	private
`projectStore`	`ProjectStore`	private
`accessPolicy`	`ProjectAccessPolicy`	private
`stateFactory`	`ProjectStateFactory`	private
`events`	`ApplicationEventPublisher`	private

Methods:

- `public ProposalService(ProposalVersionRepository versionRepository, ProposalVersionFactory versionFactory, ProjectStore projectStore, ProjectAccessPolicy accessPolicy, ProjectStateFactory stateFactory, ApplicationEventPublisher events)`
- `public ProposalVersion createInitialDraft(Project project, ProposalContent content, Actor actor)`
- `public ProposalVersion updateDraft(UUID projectId, ProposalContent content, Actor actor)`
- `public ProposalVersion createNextVersion(UUID projectId, Actor actor)`
- `public List<ProposalVersion> listVersions(UUID projectId, Actor actor)`
- `public Optional<ProposalVersion> findOpenDraft(UUID projectId)`
- `public Optional<ProposalVersion> findLatest(UUID projectId)`
- `public Optional<ProposalVersion> findLatestSubmitted(UUID projectId)`

`ProposalVersion` M1 Implemented

class `extends BaseEntity` · @Entity · `src/main/java/com/prastav/proposal/ProposalVersion.java`

Responsibility. One numbered version of a project's proposal. Business rule 2: once submitted it is never modified; #updateContent and #submit refuse to touch a SUBMITTED version, and a database trigger enforces the same rule underneath.

Field	Type	Visibility
`project`	`Project`	private
`versionNumber`	`int`	private
`problemStatement`	`String`	private
`objectives`	`String`	private
`methodology`	`String`	private
`expectedOutcome`	`String`	private
`timeline`	`String`	private

Field	Type	Visibility
`referenceLink`	`String`	private
`state`	`ProposalVersionState`	private
`submittedAt`	`Instant`	private
`createdBy`	`UUID`	private

Methods:

- `protected ProposalVersion()`
- `public ProposalVersion(Project project, int versionNumber, ProposalContent content, UUID createdBy)`
- `public void updateContent(ProposalContent content)`
- `public void submit()`
- `public boolean isDraft()`
- `public List<String> missingFields()`
- `public boolean isComplete()`
- `public ProposalContent content()`
- `public Project getProject()`
- `public int getVersionNumber()`
- `public String getProblemStatement()`
- `public String getObjectives()`
- `public String getMethodology()`
- `public String getExpectedOutcome()`
- `public String getTimeline()`
- `public String getReferenceLink()`
- `public ProposalVersionState getState()`
- `public Instant getSubmittedAt()`
- `public UUID getCreatedBy()`

`ProposalVersionCreatedEvent` M1 Implemented

record · `src/main/java/com/prastav/proposal/ProposalVersionCreatedEvent.java`

Responsibility. Observer pattern event: version N was created (v1 with the project, N+1 for a revision).

Components: `UUID projectId, int versionNumber, UUID actorId`

`ProposalVersionFactory` M1 Implemented

class · @Component · `src/main/java/com/prastav/proposal/ProposalVersionFactory.java`

Responsibility. Factory pattern. The only place that creates ProposalVersion objects. Why: version numbering (1, then previous + 1) and "prefill the revision from the previous version" are creation rules; keeping them here means they cannot be bypassed or duplicated in services.

Methods:

- `public ProposalVersion initialDraft(Project project, ProposalContent content, UUID createdBy)`
- `public ProposalVersion nextDraftFrom(ProposalVersion previous, UUID createdBy)`

`ProposalVersionRepository` **M1 Implemented**

interface `extends JpaRepository<ProposalVersion, UUID>` · `src/main/java/com/prastav/proposal/ProposalVersionRepository.java`

Responsibility. Repository pattern: persistence for ProposalVersion.

Methods:

- `List<ProposalVersion> findByProject_IdOrderByVersionNumberAsc(UUID projectId)`
- `Optional<ProposalVersion> findFirstByProject_IdOrderByVersionNumberDesc(UUID projectId)`
- `Optional<ProposalVersion> findFirstByProject_IdAndStateOrderByVersionNumberDesc(UUID projectId, ProposalVersionState state)`

`ProposalVersionState` **M1 Implemented**

enum · `src/main/java/com/prastav/proposal/ProposalVersionState.java`

Responsibility. A version is editable while DRAFT and frozen forever once SUBMITTED.

Constants: `DRAFT`, `SUBMITTED`

8.9 Package `com.prastav.proposal.web`

`ProposalController` **M1 Implemented**

class · @RestController · `src/main/java/com/prastav/proposal/web/ProposalController.java`

Responsibility. Proposal version endpoints under a project. HTTP only.

Field	Type	Visibility
`proposalService`	`ProposalService`	private

Methods:

- `public ProposalController(ProposalService proposalService)`
- `public ProposalVersionResponse updateDraft(UUID projectId, ProposalContent content, Actor actor)`
- `public ProposalVersionResponse createNextVersion(UUID projectId, Actor actor)`
- `public List<ProposalVersionResponse> list(UUID projectId, Actor actor)`

`ProposalMapper` **M1 Implemented**

final class · `src/main/java/com/prastav/proposal/web/ProposalMapper.java`

Responsibility. DTO mapper for proposal versions.

Methods:

- `public static ProposalVersionResponse toResponse(ProposalVersion version)`

`ProposalVersionResponse` **M1 Implemented**

record · `src/main/java/com/prastav/proposal/web/ProposalVersionResponse.java`

Responsibility. A proposal version as returned by the API; missingFields tells the UI what blocks submission.

Components: `UUID id, int versionNumber, ProposalVersionState state, String problemStatement, String objectives, String methodology, String expectedOutcome, String timeline, String referenceLink, List<String> missingFields, Instant submittedAt, Instant createdAt`

8.10 Package `com.prastav.review`

`ApproveDecisionHandler` M1 Implemented

class `implements ReviewDecisionHandler` · @Component · `src/main/java/com/prastav/review/ApproveDecisionHandler.java`

Responsibility. Strategy for APPROVE: moves the project UNDER_REVIEW to APPROVED. Comments are optional.

Field	Type	Visibility
`workflowService`	`WorkflowService`	private

Methods:

- `public ApproveDecisionHandler(WorkflowService workflowService)`
- `public ReviewDecision decision()`
- `public void apply(UUID projectId, Actor actor, String comments)`

`RejectDecisionHandler` M1 Implemented

class `implements ReviewDecisionHandler` · @Component · `src/main/java/com/prastav/review/RejectDecisionHandler.java`

Responsibility. Strategy for REJECT: moves UNDER_REVIEW to REJECTED (terminal). Comments are mandatory.

Field	Type	Visibility
`workflowService`	`WorkflowService`	private

Methods:

- `public RejectDecisionHandler(WorkflowService workflowService)`
- `public ReviewDecision decision()`
- `public void apply(UUID projectId, Actor actor, String comments)`

`RequestRevisionDecisionHandler` M1 Implemented

class `implements ReviewDecisionHandler` · @Component · `src/main/java/com/prastav/review/RequestRevisionDecisionHandler.java`

Responsibility. Strategy for REQUEST_REVISION: moves UNDER_REVIEW to REVISION_REQUESTED. Comments are mandatory.

Field	Type	Visibility
`workflowService`	`WorkflowService`	private

Methods:

- `public RequestRevisionDecisionHandler(WorkflowService workflowService)`

- `public ReviewDecision decision()`
- `public void apply(UUID projectId, Actor actor, String comments)`

`Review` M1 Implemented

class `extends BaseEntity` · @Entity · `src/main/java/com/prastav/review/Review.java`

Responsibility. An immutable review decision. Business rule 4: every review points at one specific submitted version, so feedback always refers to exactly the text the reviewer read.

Field	Type	Visibility
`project`	`Project`	private
`proposalVersion`	`ProposalVersion`	private
`reviewer`	`User`	private
`decision`	`ReviewDecision`	private
`comments`	`String`	private

Methods:

- `protected Review()`
- `public Review(Project project, ProposalVersion proposalVersion, User reviewer, ReviewDecision decision, String comments)`
- `public Project getProject()`
- `public ProposalVersion getProposalVersion()`
- `public User getReviewer()`
- `public ReviewDecision getDecision()`
- `public String getComments()`

`ReviewDecision` M1 Implemented

enum · `src/main/java/com/prastav/review/ReviewDecision.java`

Responsibility. The three decisions a reviewer can record.

Constants: `APPROVE`, `REQUEST_REVISION`, `REJECT`

`ReviewDecisionHandler` M1 Implemented

interface · `src/main/java/com/prastav/review/ReviewDecisionHandler.java`

Responsibility. Strategy pattern. One implementation per ReviewDecision; each one triggers the matching workflow transition. Why: ReviewService looks the handler up by decision instead of switching on it, so a new decision type is a new class and the service does not change (Open/Closed). All handlers are interchangeable behind this interface (Liskov).

Methods:

- `ReviewDecision decision()`
- `void apply(UUID projectId, Actor actor, String comments)`

`ReviewRecordedEvent` M1 Implemented

record · `src/main/java/com/prastav/review/ReviewRecordedEvent.java`

Responsibility. Observer pattern event: a review decision was stored against a specific version.

Components: `UUID projectId, UUID reviewId, int versionNumber, UUID reviewerId, ReviewDecision decision`

`ReviewRepository` **M1 Implemented**

interface `extends JpaRepository<Review, UUID>` · `src/main/java/com/prastav/review/ReviewRepository.java`

Responsibility. Repository pattern: persistence for Review.

Methods:

- `List<Review> findByProject_IdOrderByCreatedAtAsc(UUID projectId)`

`ReviewService` **M1 Implemented**

class · @Service · `src/main/java/com/prastav/review/ReviewService.java`

Responsibility. Records a reviewer's decision: picks the ReviewDecisionHandler strategy for the decision, lets it run the workflow transition (which checks status, assignment and comments), then stores the Review against the latest submitted version and publishes ReviewRecordedEvent. Everything happens in one transaction, so a failed transition stores no review.

Field	Type	Visibility
`reviewRepository`	`ReviewRepository`	private
`handlers`	`Map<ReviewDecision, ReviewDecisionHandler>`	private
`proposalService`	`ProposalService`	private
`projectStore`	`ProjectStore`	private
`accessPolicy`	`ProjectAccessPolicy`	private
`userService`	`UserService`	private
`events`	`ApplicationEventPublisher`	private

Methods:

- `public ReviewService(ReviewRepository reviewRepository, List<ReviewDecisionHandler> decisionHandlers, ProposalService proposalService, ProjectStore projectStore, ProjectAccessPolicy accessPolicy, UserService userService, ApplicationEventPublisher events)`
- `public Review recordReview(UUID projectId, ReviewDecision decision, String comments, Integer versionNumber, Actor actor)`
- `public List<Review> listReviews(UUID projectId, Actor actor)`

8.11 Package `com.prastav.review.web`

`ReviewController` **M1 Implemented**

class · @RestController · `src/main/java/com/prastav/review/web/ReviewController.java`

Responsibility. Review endpoints. HTTP only.

Field	Type	Visibility
`reviewService`	`ReviewService`	private

Methods:

- `public ReviewController(ReviewService reviewService)`
- `public ReviewResponse record(UUID projectId, ReviewRequest request, Actor actor)`
- `public List<ReviewResponse> list(UUID projectId, Actor actor)`

`ReviewMapper` M1 Implemented

final class · `src/main/java/com/prastav/review/web/ReviewMapper.java`

Responsibility. DTO mapper for reviews.

Methods:

- `public static ReviewResponse toResponse(Review review)`

`ReviewRequest` M1 Implemented

record · `src/main/java/com/prastav/review/web/ReviewRequest.java`

Responsibility. Body of POST /api/projects/{id}/reviews. Comments are required for REQUEST_REVISION and REJECT (checked by the workflow state). versionNumber is optional stale-page protection.

Components: `ReviewDecision decision, String comments, Integer versionNumber`

`ReviewResponse` M1 Implemented

record · `src/main/java/com/prastav/review/web/ReviewResponse.java`

Responsibility. A stored review decision.

Components: `UUID id, int versionNumber, ReviewDecision decision, String comments, UserSummary reviewer, Instant createdAt`

8.12 Package `com.prastav.audit`

`AuditAction` M1 Implemented

enum · `src/main/java/com/prastav/audit/AuditAction.java`

Responsibility. What happened, as recorded in audit_logs.action.

Constants: `PROJECT_CREATED`, `VERSION_CREATED`, `DRAFT_UPDATED`, `PROPOSAL_SUBMITTED`, `REVIEWER_ASSIGNED`, `REVIEW_RECORDED`, `STATUS_CHANGED`

`AuditEventListener` M1 Implemented

class · @Component · `src/main/java/com/prastav/audit/AuditEventListener.java`

Responsibility. Observer pattern. Subscribes to domain events and turns each one into an AuditLog entry. Why: the workflow, proposal and review code only publish facts; they do not know auditing exists (SRP), and a new side effect is a new listener rather than an edit to the workflow (OCP). Listeners run synchronously inside the publisher's transaction, so a change and its audit row commit or roll back together.

Field	Type	Visibility
`recorder`	`AuditRecorder`	private
`objectMapper`	`ObjectMapper`	private

Methods:

- `public AuditEventListener(AuditRecorder recorder, ObjectMapper objectMapper)`
- `public void on(ProjectCreatedEvent event)`
- `public void on(ProposalVersionCreatedEvent event)`
- `public void on(ProposalDraftUpdatedEvent event)`
- `public void on(ProjectStatusChangedEvent event)`
- `public void on(ReviewRecordedEvent event)`

`AuditLog` M1 Implemented

class `extends BaseEntity` · @Entity · `src/main/java/com/prastav/audit/AuditLog.java`

Responsibility. Append-only audit entry: who did what, when, and the status before/after. No setters; a database trigger also blocks UPDATE and DELETE on the table. Builder pattern (#builder()): an entry has up to seven fields, several optional, so a builder reads better than a long constructor and makes the optional ones explicit at the call site.

Field	Type	Visibility
`projectId`	`UUID`	private
`actorId`	`UUID`	private
`action`	`AuditAction`	private
`fromStatus`	`ProjectStatus`	private
`toStatus`	`ProjectStatus`	private
`details`	`String`	private

Methods:

- `protected AuditLog()`
- `public static Builder builder()`
- `public UUID getProjectId()`
- `public UUID getActorId()`
- `public AuditAction getAction()`
- `public ProjectStatus getFromStatus()`
- `public ProjectStatus getToStatus()`
- `public String getDetails()`

Nested `AuditLog.Builder` (static builder) methods:

- `public Builder projectId(UUID projectId)`
- `public Builder actorId(UUID actorId)`
- `public Builder action(AuditAction action)`
- `public Builder fromStatus(ProjectStatus fromStatus)`
- `public Builder toStatus(ProjectStatus toStatus)`
- `public Builder details(String details)`
- `public AuditLog build()`

`AuditLogRepository` M1 Implemented

interface `extends JpaRepository<AuditLog, UUID>` · `src/main/java/com/prastav/audit/AuditLogRepository.java`

Responsibility. Repository pattern: persistence for AuditLog. Only inserts and reads are used.

Methods:

- `List<AuditLog> findByProjectIdOrderByCreatedAtAsc(UUID projectId)`

`AuditQueryService` M1 Implemented

class · @Service · `src/main/java/com/prastav/audit/AuditQueryService.java`

Responsibility. Read side of the audit module, used by the project history timeline.

Field	Type	Visibility
`auditLogRepository`	`AuditLogRepository`	private

Methods:

- `public AuditQueryService(AuditLogRepository auditLogRepository)`
- `public List<AuditLog> findByProject(UUID projectId)`

`AuditRecorder` M1 Implemented

interface · `src/main/java/com/prastav/audit/AuditRecorder.java`

Responsibility. Small write-only port for audit entries (Interface Segregation): the listener needs to append, nothing else, so it depends on this one-method interface rather than on a repository.

Methods:

- `void record(AuditLog entry)`

`JpaAuditRecorder` M1 Implemented

class `implements AuditRecorder` · @Component · `src/main/java/com/prastav/audit/JpaAuditRecorder.java`

Responsibility. AuditRecorder backed by the audit_logs table.

Field	Type	Visibility
`auditLogRepository`	`AuditLogRepository`	private

Methods:

- `public JpaAuditRecorder(AuditLogRepository auditLogRepository)`
- `public void record(AuditLog entry)`

8.13 M2 Planned classes and members

None of the following exists in the M1 code.

New members on existing M1 classes M2 Planned

Class	Member	Purpose
`Action`	`REASSIGN_REVIEWER`	new action constant
`ProjectState`	`default void reassignReviewer(Project project, User faculty, Actor actor)`	throws by default
`UnderReviewState`	`reassignReviewer(...)` override	requires COORDINATOR, FACULTY assignee different from current; calls `project.assignReviewer`, status unchanged
`WorkflowService`	`public Project reassignReviewer(UUID projectId, UUID facultyId, Actor actor)`	facade method, publishes `ProjectStatusChangedEvent` with `from == to` and details `previousReviewerId`, `reviewerId`
`AuditAction`	`REVIEWER_REASSIGNED`	mapped from `Action.REASSIGN_REVIEWER` in `AuditEventListener.auditActionFor`
`ProjectController`	`PUT /api/projects/{id}/assign` handler `reassign(...)`	FR-17
`ProjectService`	`public Page<Project> searchProjects(ProjectSearchCriteria criteria, Actor actor, Pageable pageable)`	FR-12
`ProjectRepository`	`extends JpaSpecificationExecutor<Project>`	dynamic filters
`ProposalController`	`GET /proposals/compare` handler `compare(...)`	FR-14
`AuditQueryService`	`public Page<AuditLog> search(AuditLogFilter filter, Pageable pageable)`	FR-16

New classes M2 Planned

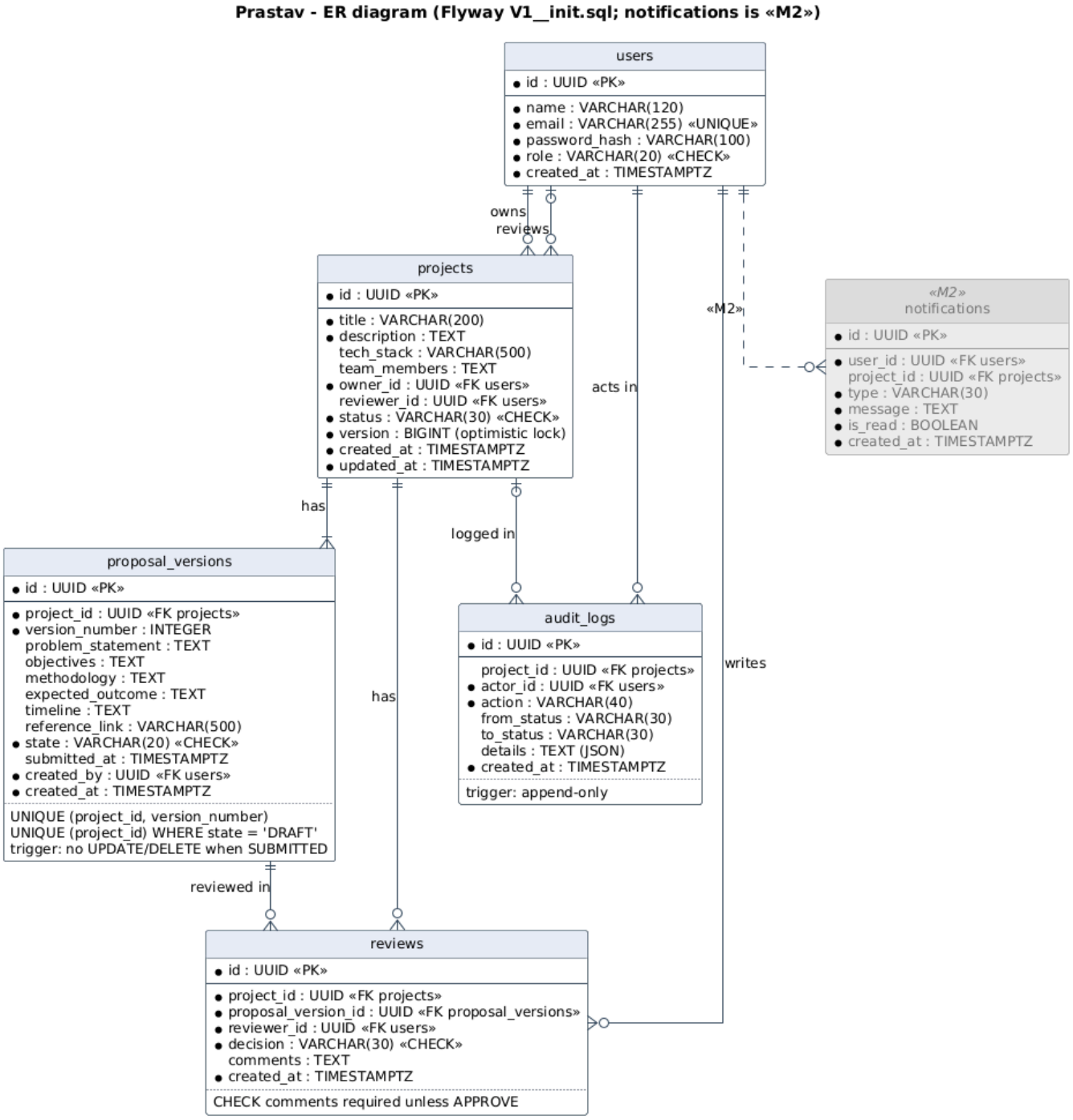
Package	Class	Kind	Fields / methods	Responsibility
notification	`Notification`	entity, extends `BaseEntity`	`User user`, `UUID projectId`, `NotificationType type`, `String message`, `boolean read`; `markRead()`	one in-app message
notification	`NotificationType`	enum	REVIEWER_ASSIGNED, REVIEW_DECISION, RESUBMITTED	what happened
notification	`NotificationRepository`	interface	`findByUser_IdOrderByCreatedAtDesc(UUID userId, Pageable p)`, `countByUser_IdAndReadFalse(UUID userId)`	persistence
notification	`NotificationChannel`	interface (Strategy)	`String name()`, `void send(Notification notification)`	delivery mechanism
notification	`InAppNotificationChannel`	`@Component`	`NotificationRepository repository`; `send` saves the row	in-app delivery
notification	`EmailNotificationChannel`	`@Component`	`send` logs only	placeholder for email

Package	Class	Kind	Fields / methods	Responsibility
notification	`NotificationService`	`@Service`	`List<NotificationChannel> channels`; `notify(User user, NotificationType type, String message)`, `listMine(Actor actor)`, `markRead(UUID id, Actor actor)`	fan-out to channels, ownership check on read
notification	`NotificationListener`	`@Component` (Observer)	`@TransactionalEventListener on(ProjectStatusChangedEvent event)`	decides who to notify
notification.web	`NotificationController`	`@RestController`	`GET /api/notifications`, `PATCH /api/notifications/{id}/read`	HTTP
notification.web	`NotificationResponse`	record	`UUID id, UUID projectId, NotificationType type, String message, boolean read, Instant createdAt`	DTO
proposal	`VersionCompareService`	`@Service`	`VersionComparison compare(UUID projectId, int from, int to, Actor actor)`	field-level diff of two versions
proposal	`VersionComparison`	record	`int fromVersion, int toVersion, List<FieldDiff> fields`	result
proposal	`FieldDiff`	record	`String field, String fromValue, String toValue, boolean changed`	one field
project	`ProjectSearchCriteria`	record	`ProjectStatus status, String query`	list filters
project	`ProjectSpecifications`	final class	`static Specification<Project> visibleTo(Actor)`, `hasStatus(ProjectStatus)`, `titleContains(String)`	composable JPA criteria
project.web	`ReassignReviewerRequest`	record	`@NotNull UUID facultyId`	body of PUT /assign
dashboard	`DashboardService`	`@Service`	`DashboardSummary summary(Actor actor)`	counts and workload queries
dashboard	`DashboardSummary`	record	`Map<ProjectStatus, Long> countsByStatus, List<ProjectSummaryResponse> pendingAssignment, List<ReviewerWorkload> workload`	response
dashboard	`ReviewerWorkload`	record	`UUID facultyId, String name, long underReview, long decided`	per reviewer
dashboard	`DashboardController`	`@RestController`	`GET /api/dashboard/summary` (COORDINATOR)	HTTP
audit	`AuditController`	`@RestController`	`GET /api/audit` (COORDINATOR)	audit viewer
audit	`AuditLogFilter`	record	`UUID projectId, UUID actorId, AuditAction action, Instant from, Instant to`	filters

Package	Class	Kind	Fields / methods	Responsibility
audit	`AuditLogResponse`	record	`UUID id, UUID projectId, UUID actorId, String actorName, AuditAction action, ProjectStatus fromStatus, ProjectStatus toStatus, String details, Instant createdAt`	DTO

9. Database schema

Owned by Flyway. M1 ships `V1__init.sql`; Hibernate runs with `ddl-auto: none`. All ids are application-generated UUIDs; all timestamps are `TIMESTAMPTZ` stored in UTC.



9.1 `users` M1 Implemented

Column	Type	Constraints
id	UUID	PK
name	VARCHAR(120)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE (stored lower-case)
password_hash	VARCHAR(100)	NOT NULL (BCrypt)
role	VARCHAR(20)	NOT NULL, CHECK in (STUDENT, FACULTY, COORDINATOR)
created_at	TIMESTAMPTZ	NOT NULL

9.2 `projects` M1 Implemented

Column	Type	Constraints
id	UUID	PK
title	VARCHAR(200)	NOT NULL
description	TEXT	NOT NULL
tech_stack	VARCHAR(500)	
team_members	TEXT	free text in V1
owner_id	UUID	NOT NULL, FK users; index
reviewer_id	UUID	FK users, nullable; index
status	VARCHAR(30)	NOT NULL, CHECK in the six statuses; index
version	BIGINT	NOT NULL DEFAULT 0, optimistic lock
created_at, updated_at	TIMESTAMPTZ	NOT NULL

9.3 `proposal_versions` M1 Implemented

Column	Type	Constraints
id	UUID	PK
project_id	UUID	NOT NULL, FK projects
version_number	INTEGER	NOT NULL, CHECK >= 1; UNIQUE (project_id, version_number)
problem_statement, objectives, methodology, expected_outcome, timeline	TEXT	nullable while DRAFT; required to submit (checked in code)
reference_link	VARCHAR(500)	optional
state	VARCHAR(20)	NOT NULL, CHECK in (DRAFT, SUBMITTED)
submitted_at	TIMESTAMPTZ	CHECK: set if and only if state = SUBMITTED
created_by	UUID	NOT NULL, FK users
created_at	TIMESTAMPTZ	NOT NULL

Extra integrity: partial unique index `ux_proposal_one_open_draft` on (project_id) WHERE state = 'DRAFT' (business rule 7); trigger `trg_proposal_versions_immutable` rejects UPDATE or DELETE of a row whose state is SUBMITTED (business rule 2).

9.4 `reviews` M1 Implemented

Column	Type	Constraints
id	UUID	PK
project_id	UUID	NOT NULL, FK projects; index
proposal_version_id	UUID	NOT NULL, FK proposal_versions
reviewer_id	UUID	NOT NULL, FK users
decision	VARCHAR(30)	NOT NULL, CHECK in (APPROVE, REQUEST_REVISION, REJECT)
comments	TEXT	CHECK: non-blank unless decision = APPROVE (business rule 5)
created_at	TIMESTAMPTZ	NOT NULL

9.5 `audit_logs` M1 Implemented

Column	Type	Constraints
id	UUID	PK
project_id	UUID	FK projects, nullable
actor_id	UUID	NOT NULL, FK users
action	VARCHAR(40)	NOT NULL (`AuditAction`)
from_status, to_status	VARCHAR(30)	nullable
details	TEXT	JSON text
created_at	TIMESTAMPTZ	NOT NULL; index (project_id, created_at)

Trigger `trg_audit_logs_append_only` rejects UPDATE and DELETE.

9.6 `notifications` M2 Planned

Migration `V2__notifications.sql`: id UUID PK; user_id UUID NOT NULL FK users; project_id UUID FK projects; type VARCHAR(30) NOT NULL; message TEXT NOT NULL; is_read BOOLEAN NOT NULL DEFAULT false; created_at TIMESTAMPTZ NOT NULL; index (user_id, is_read, created_at). M2 also adds `V3__search_indexes.sql` with a `lower(title)` index for search.

10. REST API contracts

Base path `/api`. All endpoints except register, login and Swagger need `Authorization: Bearer <jwt>`. Timestamps are ISO-8601 UTC strings. Errors always use the `ApiError` body (section 11).

Method	Path	Role	Purpose	Success	Tag
POST	/auth/register	public	Register STUDENT or FACULTY	201 `AuthResponse`	M1 Implemented

Method	Path	Role	Purpose	Success	Tag
POST	/auth/login	public	Log in	200 `AuthResponse`	**M1 Implemented**
GET	/auth/me	any	Current user	200 `UserResponse`	**M1 Implemented**
GET	/users?role=FACULTY	COORDINATOR	List users (reviewer picker)	200 `UserResponse[]`	**M1 Implemented**
POST	/projects	STUDENT	Create project + draft v1	201 `ProjectDetailResponse`	**M1 Implemented**
GET	/projects	any	Role-scoped list	200 `ProjectSummaryResponse[]`	**M1 Implemented**
GET	/projects/{id}	participant	Detail with `allowedActions`	200 `ProjectDetailResponse`	**M1 Implemented**
PUT	/projects/{id}/proposal	STUDENT owner	Edit open draft	200 `ProposalVersionResponse`	**M1 Implemented**
POST	/projects/{id}/proposals	STUDENT owner	Create next version (REVISION_REQUESTED only)	201 `ProposalVersionResponse`	**M1 Implemented**
GET	/projects/{id}/proposals	participant	All versions	200 `ProposalVersionResponse[]`	**M1 Implemented**
POST	/projects/{id}/submit	STUDENT owner	Submit or resubmit	200 `ProjectDetailResponse`	**M1 Implemented**
POST	/projects/{id}/assign	COORDINATOR	Assign reviewer	200 `ProjectDetailResponse`	**M1 Implemented**
POST	/projects/{id}/reviews	FACULTY assigned	Record decision	201 `ReviewResponse`	**M1 Implemented**
GET	/projects/{id}/reviews	participant	Review history	200 `ReviewResponse[]`	**M1 Implemented**
GET	/projects/{id}/history	participant	Merged timeline	200 `TimelineEntry[]`	**M1 Implemented**
GET	/projects?status=&q=&page=&size=	any	Filter, search, paginate	200 `Page<ProjectSummaryResponse>`	**M2 Planned**
GET	/projects/{id}/proposals/compare?from=1&to=2	participant	Field diff	200 `VersionComparison`	**M2 Planned**
PUT	/projects/{id}/assign	COORDINATOR	Reassign reviewer	200 `ProjectDetailResponse`	**M2 Planned**

Method	Path	Role	Purpose	Success	Tag
GET	/dashboard/summary	COORDINATOR	Counts, pending, workload	200 `DashboardSummary`	M2 Planned
GET	/audit	COORDINATOR	Filtered audit log	200 `Page<AuditLogResponse>`	M2 Planned
GET	/notifications	any	My notifications	200 `NotificationResponse[]`	M2 Planned
PATCH	/notifications/{id}/read	owner of notification	Mark read	204	M2 Planned

"Participant" means the owner, the assigned reviewer or any coordinator (`ProjectAccessPolicy.canView`).

10.1 Auth M1 Implemented

`POST /api/auth/register`

```
{ "name": "Aarav Mehta", "email": "aarav@prastav.edu", "password": "Password@123", "role": "STUDENT" }
```

`201 Created` (login returns the same shape with 200):

```
{
  "token":
"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI3ZjFjLi4uIiwidXNlcklkIjoiN2YxYy4uLiIsInJvbGUiOiJTVFVERU5UIn0.sig",
  "user": { "id": "7f1c2a9e-3b0d-4c1e-9a55-1d2e3f4a5b6c", "name": "Aarav Mehta", "email":
"aarav@prastav.edu", "role": "STUDENT" }
}
```

Rules: `role` COORDINATOR returns 400 VALIDATION_FAILED; a duplicate email returns 400; wrong credentials on login return 401 UNAUTHENTICATED with "Invalid email or password".

10.2 Create project M1 Implemented

`POST /api/projects` (STUDENT)

```
{
  "title": "Smart Attendance",
  "description": "Face recognition attendance for lecture halls.",
  "techStack": "Java, React, OpenCV",
  "teamMembers": "Aarav Mehta, Diya Kapoor",
  "proposal": {
    "problemStatement": "Attendance is taken on paper and takes ten minutes per lecture.",
    "objectives": "Automate attendance using face recognition.",
    "methodology": null,
    "expectedOutcome": null,
    "timeline": null,
    "referenceLink": null
  }
}
```

`201 Created`, `ProjectDetailResponse`:

```
{
  "id": "0b8f6c1e-2d4a-4f7b-8e3c-5a6b7c8d9e0f",
  "title": "Smart Attendance",
  "description": "Face recognition attendance for lecture halls.",
  "techStack": "Java, React, OpenCV",
  "teamMembers": "Aarav Mehta, Diya Kapoor",
  "status": "DRAFT",
  "owner": { "id": "7f1c2a9e-3b0d-4c1e-9a55-1d2e3f4a5b6c", "name": "Aarav Mehta" },
  "reviewer": null,
  "currentProposal": {
    "id": "c3d4e5f6-0000-4a1b-9c2d-3e4f5a6b7c8d",
    "versionNumber": 1,
    "state": "DRAFT",
    "problemStatement": "Attendance is taken on paper and takes ten minutes per lecture.",
    "objectives": "Automate attendance using face recognition.",
    "methodology": null,
    "expectedOutcome": null,
    "timeline": null,
    "referenceLink": null,
    "missingFields": ["methodology", "expectedOutcome", "timeline"],
    "submittedAt": null,
    "createdAt": "2026-10-19T09:00:00.120Z"
  },
  "latestSubmittedVersion": null,
  "allowedActions": ["SUBMIT", "EDIT_DRAFT"],
  "version": 0,
  "createdAt": "2026-10-19T09:00:00.118Z",
  "updatedAt": "2026-10-19T09:00:00.118Z"
}
```


`allowedActions` is computed for the caller: the same project fetched by the coordinator returns `[]` while DRAFT and `["ASSIGN_REVIEWER"]` once SUBMITTED.

10.3 Edit draft and versions **M1 Implemented**

`PUT /api/projects/{id}/proposal` with a full `ProposalContent` body (same six fields as above) returns `ProposalVersionResponse`. In SUBMITTED or UNDER_REVIEW it returns 409 `ILLEGAL_TRANSITION` "Action EDIT_DRAFT is not allowed while the project is SUBMITTED". In REVISION_REQUESTED before a new version exists it returns 409 "There is no open draft; create a new version first".

`POST /api/projects/{id}/proposals` (no body) returns `201` with version N+1 in state DRAFT, prefilled from version N. A second call while that draft is open returns 409 "Version 2 is already an open draft".

`GET /api/projects/{id}/proposals` returns all versions oldest first; submitted versions have `"state": "SUBMITTED"` and a `submittedAt`.

10.4 Submit and assign **M1 Implemented**

`POST /api/projects/{id}/submit` (no body) returns the updated `ProjectDetailResponse` (`"status": "SUBMITTED"` from DRAFT, or `"UNDER_REVIEW"` from REVISION_REQUESTED). An incomplete draft returns:


```
{
  "timestamp": "2026-10-19T09:05:11.402Z",
  "status": 400,
  "code": "VALIDATION_FAILED",
  "message": "Proposal is incomplete. Missing: methodology, expectedOutcome, timeline",
  "path": "/api/projects/0b8f6c1e-2d4a-4f7b-8e3c-5a6b7c8d9e0f/submit",
  "fieldErrors": []
}
```


`POST /api/projects/{id}/assign` (COORDINATOR)

```
{ "facultyId": "a1b2c3d4-1111-4a2b-8c3d-4e5f6a7b8c9d" }
```

returns `ProjectDetailResponse` with `"status": "UNDER_REVIEW"` and `"reviewer": { "id": "a1b2c3d4-...", "name": "Dr. Anita Rao" }`. A non-faculty `facultyId` returns 400 "Reviewer must have the FACULTY role".

10.5 Reviews M1 Implemented

`POST /api/projects/{id}/reviews` (assigned FACULTY)

```
{ "decision": "REQUEST_REVISION", "comments": "Explain how student consent and data retention are
handled.", "versionNumber": 1 }
```

`201 Created`, `ReviewResponse`:

```
{
  "id": "e9f8a7b6-2222-4c3d-8e4f-5a6b7c8d9e0f",
  "versionNumber": 1,
  "decision": "REQUEST_REVISION",
  "comments": "Explain how student consent and data retention are handled.",
  "reviewer": { "id": "a1b2c3d4-1111-4a2b-8c3d-4e5f6a7b8c9d", "name": "Dr. Anita Rao" },
  "createdAt": "2026-10-19T10:12:40.017Z"
}
```

`versionNumber` is optional; if sent and not the latest submitted version: 409 "Only the latest submitted version (v2) can be reviewed". Missing comments for REQUEST_REVISION or REJECT: 400 "Comments are mandatory for REQUEST_REVISION". An unassigned faculty member: 403 "Only the assigned reviewer can review this project". Approving a SUBMITTED project:

```
{
  "timestamp": "2026-10-19T10:01:02.551Z",
  "status": 409,
  "code": "ILLEGAL_TRANSITION",
  "message": "Action APPROVE is not allowed while the project is SUBMITTED",
  "path": "/api/projects/0b8f6c1e-2d4a-4f7b-8e3c-5a6b7c8d9e0f/reviews",
  "fieldErrors": []
}
```

10.6 History M1 Implemented

`GET /api/projects/{id}/history` returns `TimelineEntry[]` sorted by time:

```
[
  { "at": "2026-10-19T09:00:00.118Z", "type": "STATUS_CHANGED", "actorId": "7f1c...", "actorName": "Aarav
Mehta", "summary": "Project created as DRAFT", "versionNumber": null },
  { "at": "2026-10-19T09:00:00.120Z", "type": "VERSION_CREATED", "actorId": "7f1c...", "actorName": "Aarav
Mehta", "summary": "Version 1 created", "versionNumber": 1 },
  { "at": "2026-10-19T09:20:31.004Z", "type": "STATUS_CHANGED", "actorId": "7f1c...", "actorName": "Aarav
Mehta", "summary": "DRAFT -> SUBMITTED", "versionNumber": null },
  { "at": "2026-10-19T09:41:12.660Z", "type": "STATUS_CHANGED", "actorId": "5d6e...", "actorName": "Meera
Nair", "summary": "SUBMITTED -> UNDER_REVIEW", "versionNumber": null },
  { "at": "2026-10-19T10:12:40.010Z", "type": "STATUS_CHANGED", "actorId": "a1b2...", "actorName": "Dr.
Anita Rao", "summary": "UNDER_REVIEW -> REVISION_REQUESTED", "versionNumber": null },
  { "at": "2026-10-19T10:12:40.017Z", "type": "REVIEW", "actorId": "a1b2...", "actorName": "Dr. Anita Rao",
"summary": "REQUEST_REVISION on version 1: Explain how student consent and data retention are handled.",
"versionNumber": 1 }
]
```

10.7 M2 Planned contracts

`GET /api/projects/{id}/proposals/compare?from=1&to=2`

```
{ "fromVersion": 1, "toVersion": 2, "fields": [
  { "field": "methodology", "fromValue": "Spring Boot API, React UI.", "toValue": "Spring Boot API, React
UI; consent forms and 30-day retention.", "changed": true },
  { "field": "timeline", "fromValue": "8 weeks", "toValue": "8 weeks", "changed": false } ] }
```

`GET /api/dashboard/summary`

```
{ "countsByStatus": { "DRAFT": 3, "SUBMITTED": 2, "UNDER_REVIEW": 4, "REVISION_REQUESTED": 1, "APPROVED":
5, "REJECTED": 1 },
  "pendingAssignment": [ { "id": "...", "title": "Library Seat Finder", "status": "SUBMITTED", "owner": {
"id": "...", "name": "Diya Kapoor" }, "reviewer": null, "updatedAt": "..." } ],
  "workload": [ { "facultyId": "...", "name": "Dr. Anita Rao", "underReview": 3, "decided": 6 } ] }
```

`GET /api/notifications`

```
[ { "id": "...", "projectId": "...", "type": "REVIEW_DECISION", "message": "Dr. Anita Rao requested a
revision of Smart Attendance (v1).", "read": false, "createdAt": "..." } ]
```

`GET /api/audit?projectId=&actorId=&action=&from=&to=&page=0&size=20` returns a Spring `Page` of `AuditLogResponse`. `PUT /api/projects/{id}/assign` takes `{ "facultyId": "..." }` like POST and keeps the status UNDER_REVIEW.

11. Validation and error codes

`ApiError` M1 Implemented: `{ timestamp, status, code, message, path, fieldErrors[] }` with `fieldErrors` items `{ field, message }`.

Code	HTTP	Raised by	Example
VALIDATION_FAILED	400	`@Valid` failures (with `fieldErrors`), malformed JSON or UUID, `ValidationException` (incomplete proposal, missing review comments, non-faculty assignee, coordinator self-registration, duplicate email)	`fieldErrors: [{ "field": "title", "message": "must not be blank" }]`
UNAUTHENTICATED	401	`RestAuthErrorHandler.commence` (no/invalid/expired token), `UnauthenticatedException` (bad login)	"Authentication required: send a valid Bearer token"
FORBIDDEN	403	`@PreAuthorize` denial, `ForbiddenException` from `ProjectAccessPolicy` or `Guards`	"Only the assigned reviewer can review this project"
NOT_FOUND	404	`NotFoundException`	"Project 0b8f... not found"
ILLEGAL_TRANSITION	409	`IllegalTransitionException`, `ConflictException` (edit submitted version, second open draft, stale versionNumber, resubmit without new version)	"Action APPROVE is not allowed while the project is SUBMITTED"
CONCURRENT_UPDATE	409	`OptimisticLockingFailureException` on `Project.version`	"The project was changed by someone else at the same time. Reload and try again."

Code	HTTP	Raised by	Example
INTERNAL_ERROR	500	anything unexpected (logged with stack trace, never returned)	"Unexpected server error"

Bean Validation limits: title 200, description 4000, techStack 500, teamMembers 2000, each proposal text field 4000 (timeline 2000), referenceLink 500, review comments 4000, password 8 to 72 characters (BCrypt input limit). Draft fields may be blank; completeness is enforced on submit by `ProposalVersion.missingFields()`.

12. Security design

Concern	Design
Passwords	BCrypt (`BCryptPasswordEncoder`), never returned (`UserResponse` has no hash)
Tokens	HS256 JWT, 60-minute expiry, claims `userId` and `role` (+ `sub`), secret from `JWT_SECRET` (startup fails if shorter than 32 bytes)
Sessions	Stateless (`SessionCreationPolicy.STATELESS`), CSRF disabled because no cookies are used
Authorization layer 1	`@PreAuthorize("hasRole(...)")` on controller methods: STUDENT for create/edit/submit/revise, COORDINATOR for assign and user list, FACULTY for reviews
Authorization layer 2	Ownership and assignment in the service/domain: `ProjectAccessPolicy` for reads and proposal edits, `Guards` inside state transitions. Needed because a role check cannot tell which student owns which project.
Visibility	Business rule 1: `ProjectService.listProjects` scopes queries by role; `requireView` on every per-project read
Errors	401/403 from the filter chain are written by `RestAuthErrorHandler` in `ApiError` format; no stack traces in responses
CORS	Only `prastav.cors.allowed-origin` (default `http://localhost:5173`) for `/api/**`; in development Vite proxies `/api`, so the browser uses one origin
Registration	Self-registration limited to STUDENT and FACULTY; the coordinator account comes from seed data
Frontend	Token in `localStorage` (acceptable for an intranet demo; M2 hardening option: httpOnly cookie). A 401 response clears the session. Route guards hide pages by role; the backend remains the authority.

13. Design patterns

Pattern	Where (exact classes)	Why
State	`ProjectState`, `DraftState`, `SubmittedState`, `UnderReviewState`, `RevisionRequestedState`, `ApprovedState`, `RejectedState`	Each status owns its legal actions; no status switch statements; illegal by default
Factory	`ProjectStateFactory` (status to state), `ProposalVersionFactory` (v1 and vN+1 prefilled)	Creation and lookup rules live in one place
Facade	`WorkflowService`	One simple API over states, persistence and events; every transition takes the same audited path

Pattern	Where (exact classes)	Why
Observer	`ProjectCreatedEvent`, `ProposalVersionCreatedEvent`, `ProposalDraftUpdatedEvent`, `ProjectStatusChangedEvent`, `ReviewRecordedEvent` consumed by `AuditEventListener`; [M2] `NotificationListener`	Side effects decoupled from the workflow
Strategy	`ReviewDecisionHandler` + `ApproveDecisionHandler`, `RequestRevisionDecisionHandler`, `RejectDecisionHandler`; [M2] `NotificationChannel` + `InAppNotificationChannel`, `EmailNotificationChannel`	Add a decision or channel by adding a class
Repository	`UserRepository`, `ProjectRepository`, `ProposalVersionRepository`, `ReviewRepository`, `AuditLogRepository`	Persistence abstraction (Spring Data JPA)
Builder	`AuditLog.builder()` / `AuditLog.Builder`	Readable construction with optional fields
DTO + Mapper	`*Request` / `*Response` records, `UserMapper`, `ProjectMapper`, `ProposalMapper`, `ReviewMapper`	Entities never leave the service layer as JSON
Value object	`ProposalContent`, `Actor`, `NewProject`	Immutable records passed between layers

14. SOLID mapping

Principle	Evidence in code
Single responsibility	`ProjectController` handles HTTP only; `ProjectService` handles project use cases; `DraftState` handles only DRAFT rules; `AuditEventListener` handles only auditing; `ProposalVersionFactory` only creates versions; `AllowedActionsResolver` only computes buttons
Open/closed	New status: new `ProjectState` class (auto-registered by `ProjectStateFactory`). New decision: new `ReviewDecisionHandler`. New side effect: new `@EventListener` class. [M2] new channel: new `NotificationChannel`. None needs edits to `WorkflowService` or `ReviewService`
Liskov substitution	Any `ProjectState` can be returned by the factory and used by `WorkflowService` the same way; every `ReviewDecisionHandler` honours the same `apply` contract; `TerminalStatesTest` exercises states through the interface
Interface segregation	`AuditRecorder` has one method; `ReviewDecisionHandler` has two; `ProjectState` holds only transition methods (no persistence); `ProjectStore` exposes only `getById` / `save` to other modules
Dependency inversion	Services receive collaborators through constructors and depend on abstractions (`ApplicationEventPublisher`, `AuditRecorder`, `List<ProjectState>`, `List<ReviewDecisionHandler>`, `PasswordEncoder`). State classes and entities import no Spring service or repository, so they are tested with plain `new`

15. Design decisions and trade-offs

15.1 Decisions

Decision	Alternatives	Why this one	Cost
Modular monolith	Microservices	One team, one database, simple transactions; modules can be split later along the same boundaries	Discipline needed to respect module boundaries
State pattern	Enum + switch	Open/closed, one class per status, isolated tests	More classes; reading the flow means opening several files (mitigated by the diagrams)
Synchronous `@EventListener` for audit	`@TransactionalEventListener(AFTER_COMMIT)`, outbox	Audit must be exactly as durable as the change: same transaction	A failing audit insert rolls back the business change (acceptable: audit is mandatory)
Immutable versions + DB trigger	Edit in place with history table	Reviews must judge exactly the submitted text; defence in depth	Each revision duplicates text (tiny)
Single reviewer per project	Reviewer panel	PRD V1 scope	Two-reviewer rule needs a policy change (see INTERVIEW_PREP)
Proposal as text fields + link	File upload	No file storage in V1; diffable fields for M2 compare	Formatting is plain text
Application-assigned UUIDs + `Persistable`	DB sequences	Entities have ids before saving; pure unit tests	Must implement `isNew()` correctly (done in `BaseEntity`)
Optimistic locking on `Project`	Pessimistic locks	Conflicts are rare; no lock waits	Caller sees 409 and retries
`allowedActions` from backend	UI computes from status	One source of truth; UI cannot drift	One more field per detail response
Comments check in state and DB	Only in code	Business rule 5 enforced even for direct SQL	Two places to change if the rule changes
`ddl-auto: none`	`validate`	Flyway is the single source of truth; avoids `TEXT` vs `VARCHAR` false alarms	Mapping errors surface at first query, not at startup
`open-in-view: false`, eager owner/reviewer	OSIV	No lazy loading from controllers	A few extra joins/selects on lists (fine for demo data; M2 pagination uses entity graphs)
REJECTED is final	Coordinator reopen	PRD V1	A reopen would be a new state/transition
Coordinator not self-registered	Open registration	Prevent privilege escalation	Coordinator created by seed data

15.2 Known limitations in M1

The project list is not paginated (M2). The timeline is API-only (UI tab is M2). Notifications, compare, reassign, dashboard and audit viewer are M2. Frontend has no automated tests. The JWT is stored in `localStorage`.

15.3 M2 Planned DevOps design

`backend/Dockerfile`: multi-stage, `maven:3.9-eclipse-temurin-17` build, `eclipse-temurin:17-jre` runtime, non-root user. `frontend/Dockerfile`: Node 20 build, `nginx:alpine` serving `dist` with `/api` proxied to the backend (`frontend/nginx.conf`). `docker-compose.yml` gains `backend` and `frontend` services with `depends_on: db: condition: service_healthy`, so `docker compose up` starts everything with seed data.

`.github/workflows/ci.yml` on pull requests and pushes to `main` / `develop`: job backend (`mvn -B verify` with JaCoCo report uploaded as an artifact), job frontend (`npm ci`, `npm run lint`, `npm run build`). Branch protection on `main` requires green CI and one review.

16. Testing design

16.1 M1 Implemented tests (`backend/src/test/java`)

Test class	What it proves
`DraftStateTest`	owner submits complete draft (version frozen); non-owner 403; incomplete 400; no draft 409; assign/approve/revise/reject illegal
`SubmittedStateTest`	coordinator assigns faculty; non-coordinator 403; non-faculty assignee 400; approving a SUBMITTED project is illegal
`UnderReviewStateTest`	approve, request revision, reject; comments mandatory; unassigned faculty and student 403; submit/assign illegal
`RevisionRequestedStateTest`	resubmit returns to UNDER_REVIEW with same reviewer; no new version 409; non-owner 403; review/assign illegal
`TerminalStatesTest`	APPROVED and REJECTED refuse every action through the interface
`ProjectStateFactoryTest`	every status resolves; missing or duplicate state fails fast
`AllowedActionsResolverTest`	buttons per role and state, including CREATE_REVISION vs EDIT_DRAFT
`WorkflowServiceTest`	transitions save and publish the event with from/to/details; illegal transition saves and publishes nothing; revision loop keeps the reviewer
`ReviewServiceTest`	strategy lookup per decision; review stored against latest submitted version; stale version 409 before any transition; failed transition stores nothing; non-participant 403
`ProposalServiceTest`	sequential numbering and prefill; v1 unchanged after v2 edits; one open draft; no revision while UNDER_REVIEW; no edit when SUBMITTED; owner-only edit
`ProposalVersionTest`	immutability; missing-field detection; factory numbering
`ProjectServiceTest`	create in DRAFT with v1 and event; faculty cannot create; role-scoped lists; stranger 403; latest submitted version in detail
`AuditEventListenerTest`	status change becomes an audit row with before/after and JSON details; action mapping
`ProjectControllerWebTest` (MockMvc, `@WebMvcTest`)	401 without token; 403 for a student calling assign; 409 body for an illegal transition

All unit tests are plain JUnit 5 + Mockito with real state objects; no Spring context and no database are needed.

16.2 M2 Planned tests

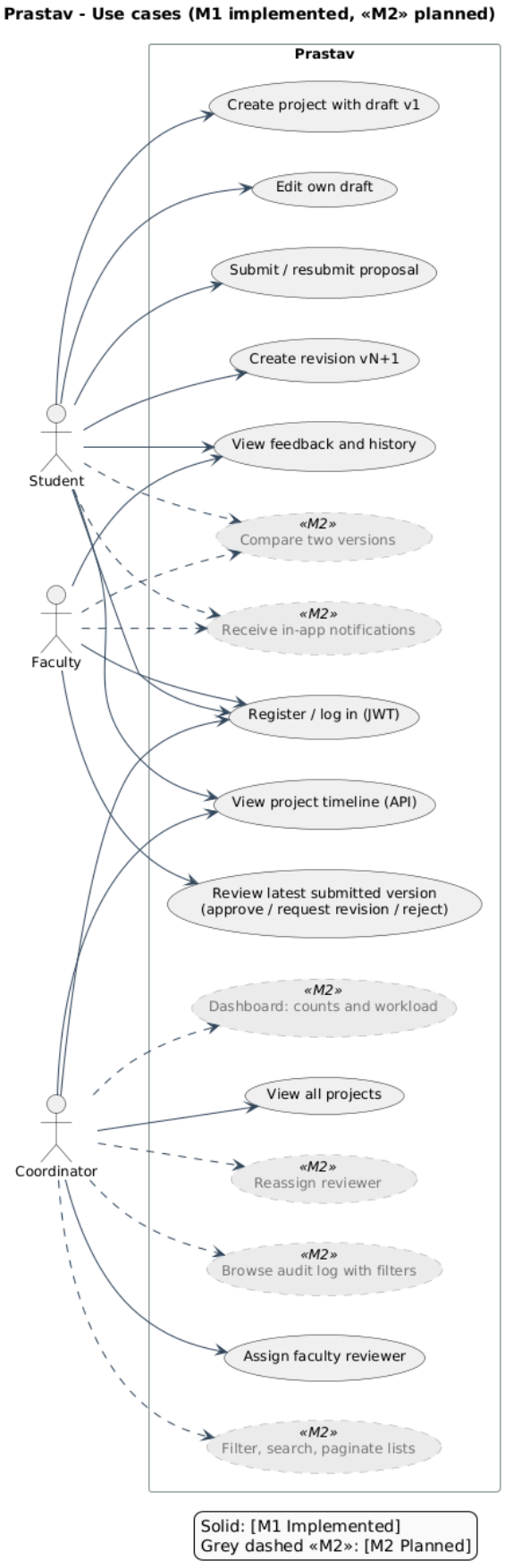
`WorkflowIntegrationTest` (Testcontainers PostgreSQL, `@SpringBootTest`): full happy path and the revision loop through HTTP, asserting audit rows and the immutability trigger.
`ConcurrentSubmissionIntegrationTest`: two threads submit the same project; exactly one succeeds and the other gets `CONCURRENT_UPDATE` or `ILLEGAL_TRANSITION`. Controller slice tests for every controller.
JaCoCo target of about 70 percent line coverage, reported in CI.

17. Diagram index

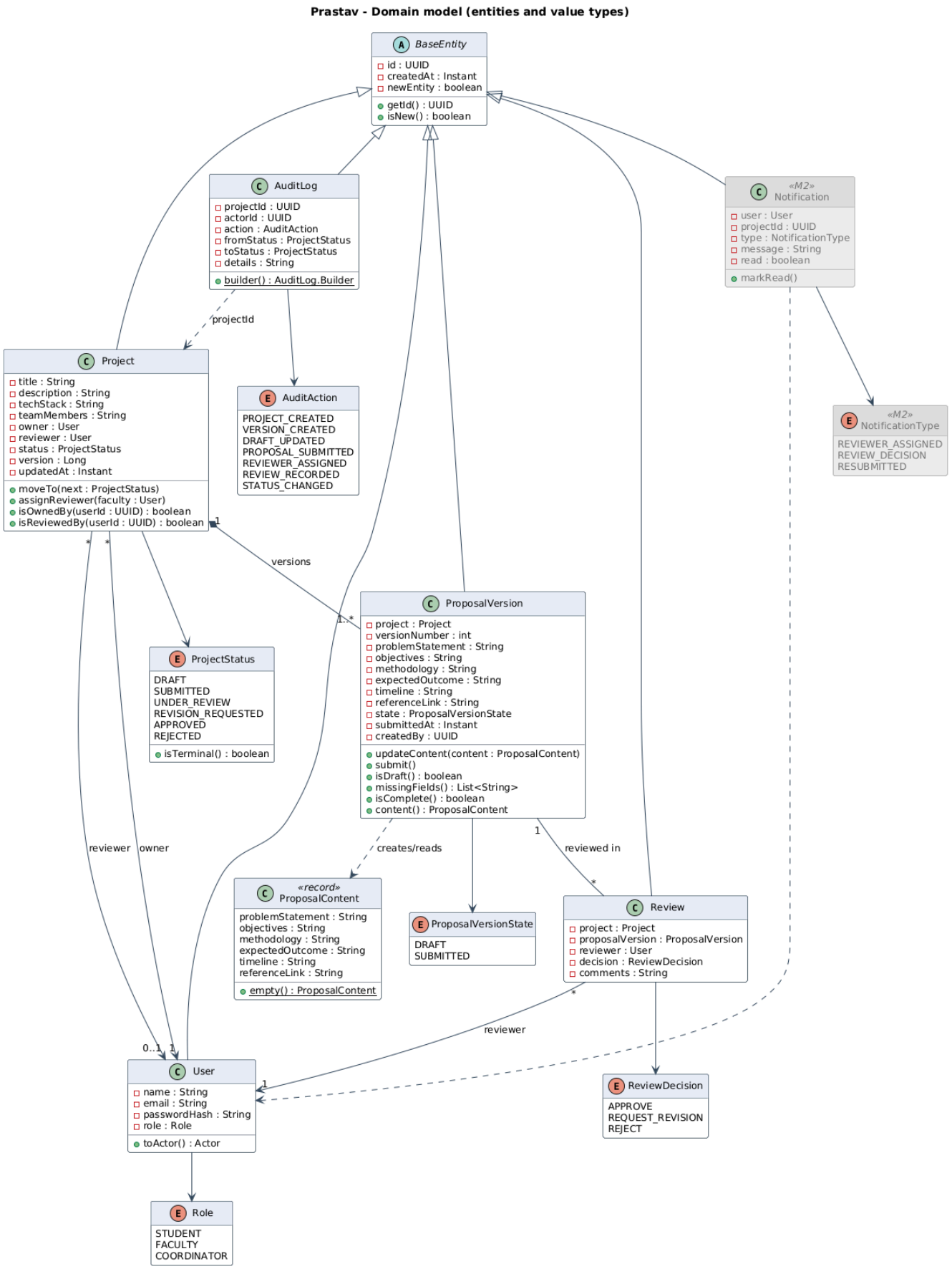
PlantUML sources are in `docs/uml/*.puml` (shared style `_style.iuml`), rendered PNGs in `docs/uml/png/`. Mermaid equivalents are in `docs/diagrams.md` with renders in `docs/uml/mermaid-png/`.

#	Diagram	File
1	Use cases	`01_use_case`
2	Domain class diagram	`02_domain_class`
3	Workflow State-pattern class diagram	`03_workflow_state_pattern`
4	Project state diagram with guards	`04_project_state`
5	Sequence: create + submit	`05_seq_create_submit`
6	Sequence: assign + review + request revision	`06_seq_assign_review_revision`
7	Sequence: resubmit + approve	`07_seq_resubmit_approve`
8	Sequence: illegal transition rejected	`08_seq_illegal_transition`
9	Sequence: [M2] notification flow	`09_seq_m2_notification`
10	ER diagram	`10_er`
11	Component / architecture	`11_component_architecture`
12	Observer and Strategy	`12_events_and_strategies`

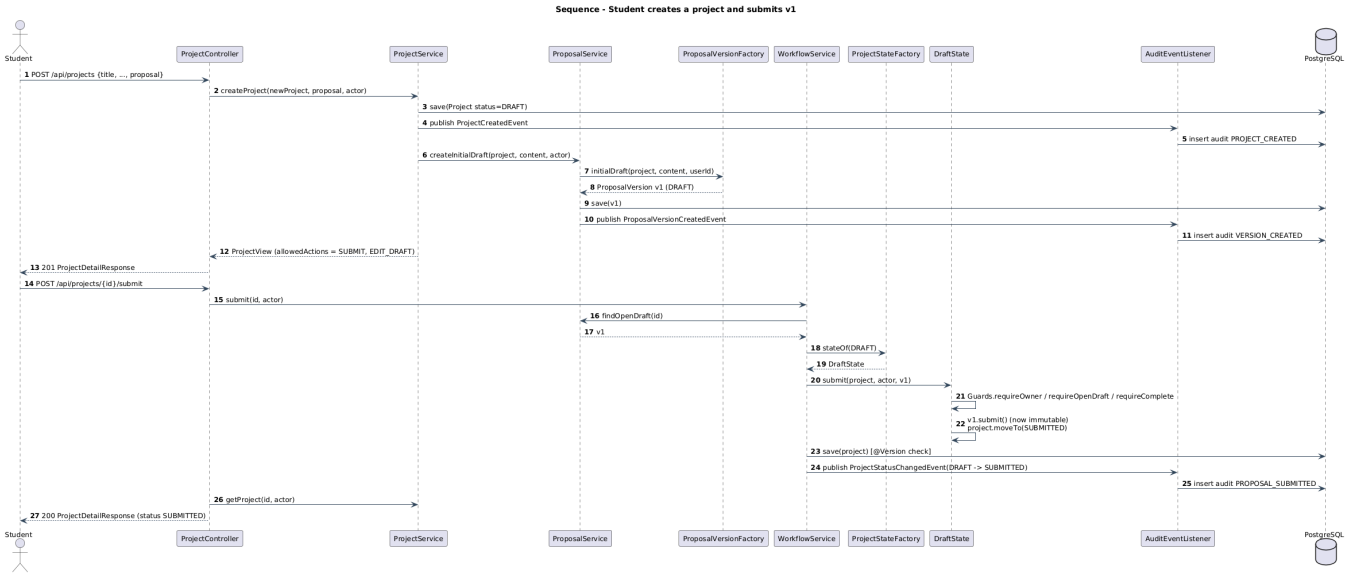
Use cases


Prastav - Use cases (M1 implemented, «M2» planned)
Prastav
Create project with draft v1
Edit own draft
Submit / resubmit proposal
Create revision vN+1
View feedback and history
«M2»
Compare two versions
«M2»
Receive in-app notifications
Register / log in (JWT)
View project timeline (API)
Review latest submitted version
(approve / request revision / reject)
«M2»
Dashboard: counts and workload
View all projects
«M2»
Reassign reviewer
«M2»
Browse audit log with filters
Assign faculty reviewer
«M2»
Filter, search, paginate lists
Student
Faculty
Coordinator
Solid: [M1 Implemented]
Grey dashed «M2»: [M2 Planned]

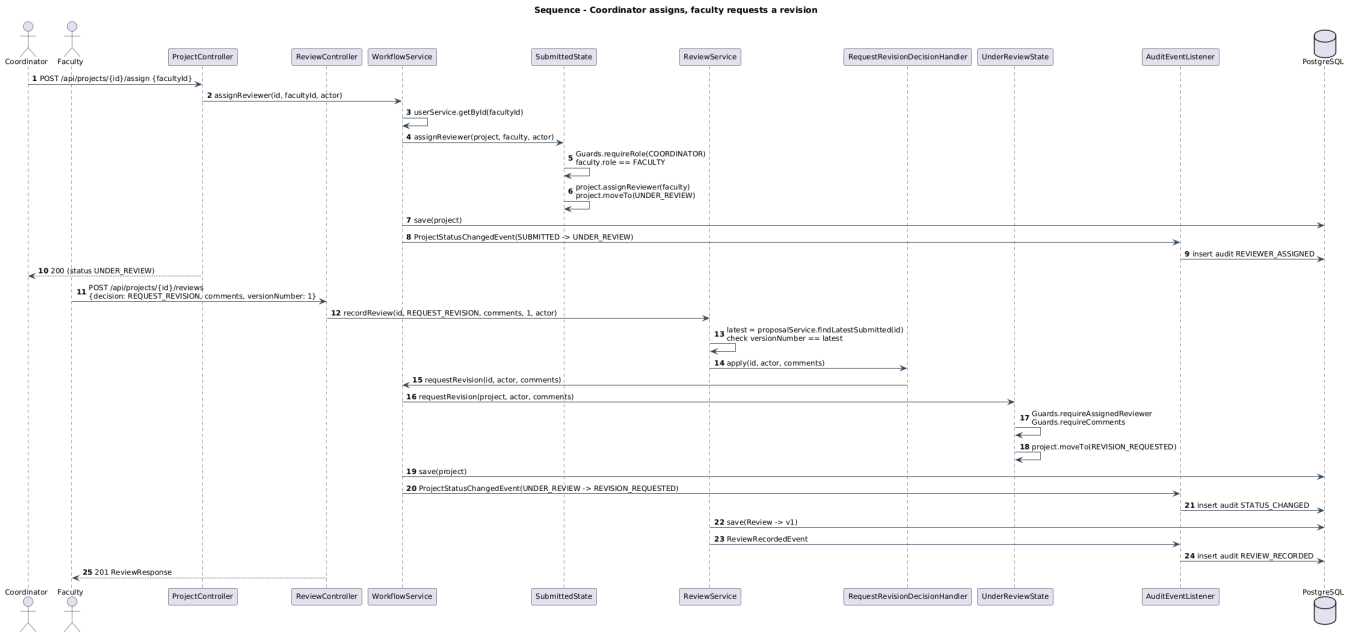
Domain model



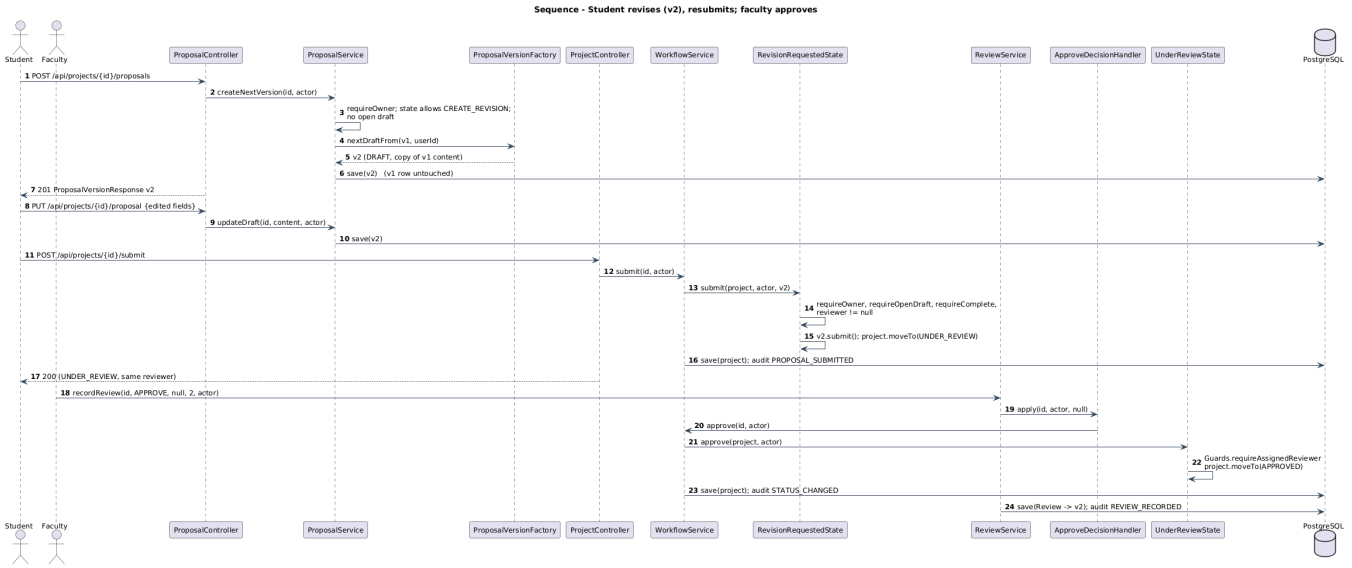
Sequence: create and submit



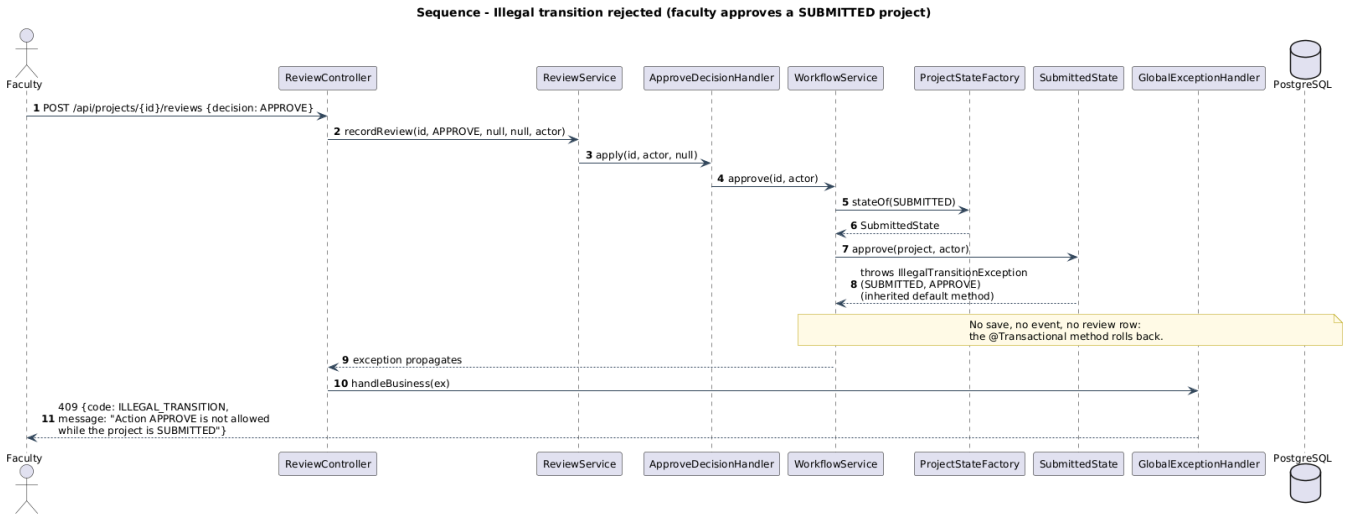
Sequence: assign, review, request revision



Sequence: resubmit and approve



Sequence: illegal transition



Sequence: M2 Planned notification

